

Lekiwi 환경 구축 및 원격조종 물체 스폰 매뉴얼

0. 실행환경

- Ubuntu 22.04
- Python 3.10.0
- Isaac Sim 5.0.0
- Lerobot 0.4.3
- ROS2 Humble
- CUDA + PyTorch
- Conda

1. 운영 아키텍처

[실기/리더 제어 PC]

리더암 입력 + 키보드 base 명령 -> TCP JSON 송신

[Home/시물 PC]

TCP 수신 -> ROS2 JointState/휠명령 publish -> Isaac Sim 반영

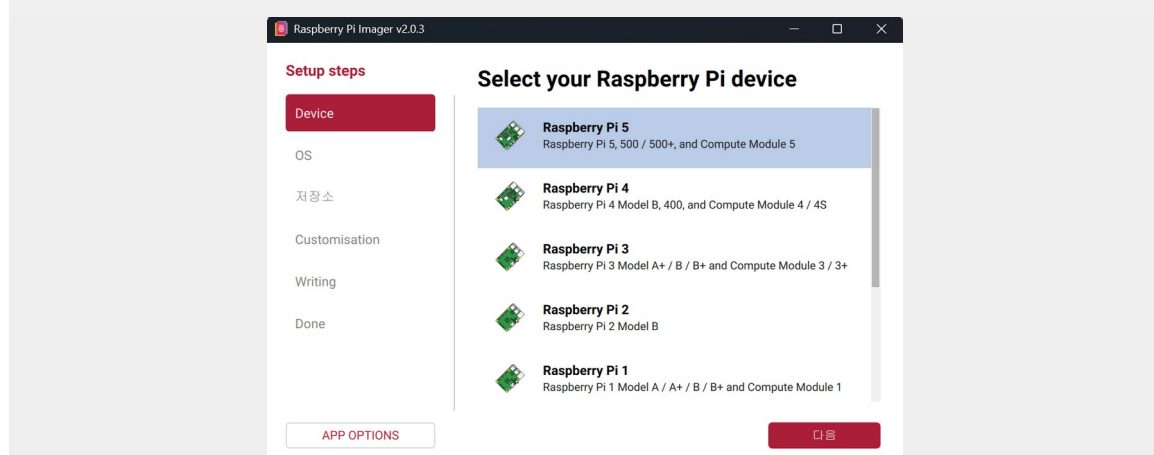
2. Lekiwi 실물 로봇 세팅

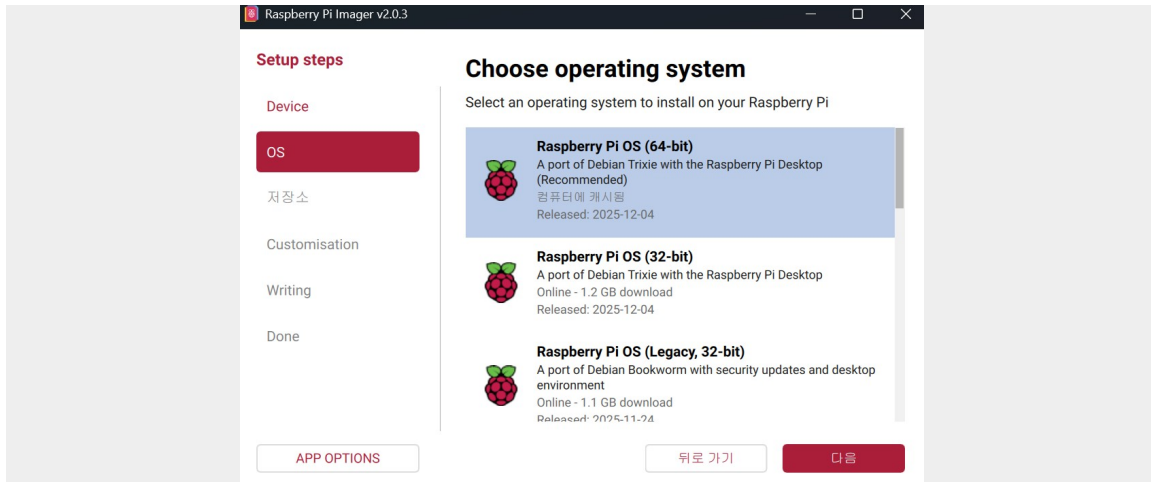
2.0 라즈베리파이 소프트웨어 설치

Lekiwi 로봇을 세팅하기 위해서는 로봇을 구동할 라즈베리파이에 OS를 먼저 설치해야 한다

<https://www.raspberrypi.com/software/>

위 링크의 Raspberry Pi Imager를 통해 SD 카드에 Raspberry PI OS를 설치한다





OS 설치 이후 원활한 원격 접속을 위해 Raspberry pi connect 에 로봇을 등록하거나 ssh <사용자명><ip 주소>로 로봇을 개인 환경에 등록한다
이후 라즈베리파이에서 Lerobot 을 설치하기 위해 미니콘다 가상환경과 Lerobot 레퍼지토리를 깃클론한다

```
# 미니콘다 설치
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-aarch64.sh
bash Miniconda3-latest-Linux-aarch64.sh
source ~/.bashrc
```

```
# 가상환경 생성
conda create -y -n lerobot python=3.10
conda activate lerobot
```

```
# ffmpeg 설치
conda install ffmpeg -c conda-forge
```

```
# LeRobot 클론 및 설치
git clone https://github.com/huggingface/lerobot.git
cd lerobot
git checkout v0.4.3
pip install -e ".[lekiwi]"
```

2.1 로봇 조립 및 세팅

로봇 구성 : <https://partabot-1.gitbook.io/partabot/lekiwi>
로봇 조립 : <https://www.youtube.com/watch?v=cKWAjEV4aSg>
로봇 연결 : <https://huggingface.co/docs/lerobot/lekiwi>

2.2 로봇 포트 인식

로봇이 노트북 혹은 데스크탑에 연결된 포트를 찾는 과정으로 진행 후 이를 기록해 추후에 사용한다

ex)

```
lerobot-find-port
```

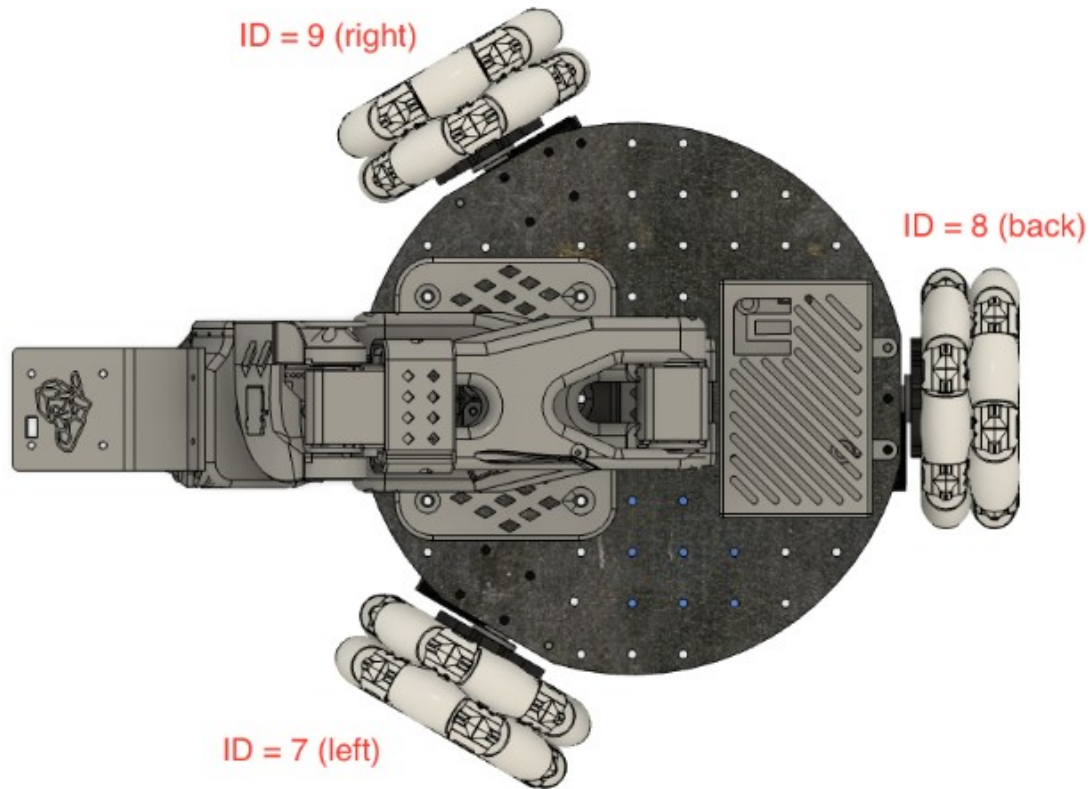
2.3 모터 번호 인식

모터 번호 인식은 사용하는 모터보드에 연결된 모터의 번호를 인식시켜주는 과정으로 모터 보드 교체 혹은 모터 교체시마다 진행하는 것이 안전하다

Lekiwi 로봇은 모터 ID 1~6 은 ARM, 7~9 는 Wheel 이다

ex)

```
lerobot-setup-motors \\  
--robot.type=lekiwi \\  
--robot.port=/dev/ttyACM0
```



2.4 로봇 캘리브레이션

로봇의 물리적 값과 소프트웨어 상의 값을 일치시키는 과정으로 사용할 리더암과 팔로워암의 초기 자세를 동일하게 설정하고 진행해야 한다

ex)

Lekiwi 팔로워암 -> 라즈베리파이에서 실행

```
lerobot-calibrate \\  
--robot.type=lekiwi \\  
--robot.id=my_awesome_kiwi
```

리더암 -> 노트북이나 데스크탑에서 실행

```
lerobot-calibrate \\  
--teleop.type=so100_leader \\  
--teleop.port=/dev/ttyACM0 \\  
--teleop.id=my_awesome_leader_arm
```

2.5 로봇 텔레옴

캘리브레이션이 정상적으로 진행되었는지 확인하는 단계로 인자는 리더암/팔로워암의 로봇 타입, 포트 번호, 캘리브레이션 파일명으로 진행된다

이때 라즈베리파이에 원격으로 신호를 보내기 위해 노트북이나 데스크탑 환경의 examples/lekiwi/teleoperate.py 파일에서 remote_ip 를 라즈베리파이의 ip 로, port 를 리더암이 연결된 포트 번호로 변경 후 진행한다

조작키

W	Move forward
A	Move left
S	Move backward
D	Move right
Z	Turn left
X	Turn right
R	Increase speed
F	Decrease speed

ex)

라즈베리파이

```
python -m lerobot.robots.lekiwi.lekiwi_host --robot.id=my_awesome_kiwi
```

노트북 / 데스크탑

```
python examples/lekiwi/teleoperate.py
```

3. Issac sim Lekiwi 로봇 세팅

3.0 Issac sim 설치 및 Lekiwi 로봇 usd 파일 설치

Isaac sim 설치

<https://docs.isaacsim.omniverse.nvidia.com/5.0.0/installation/download.html> 이 링크에 접속해 isaac sim 5.0.0 ubuntu 버전을 다운로드 해준다

이후 설치한 파일을 압축 해제 후 설치해준다

```
mkdir ~/isaacsim
cd ~/Downloads
unzip "isaac-sim-standalone-5.0.0-linux-x86_64.zip" -d ~/isaacsim
cd ~/isaacsim
./post_install.sh
```

실행

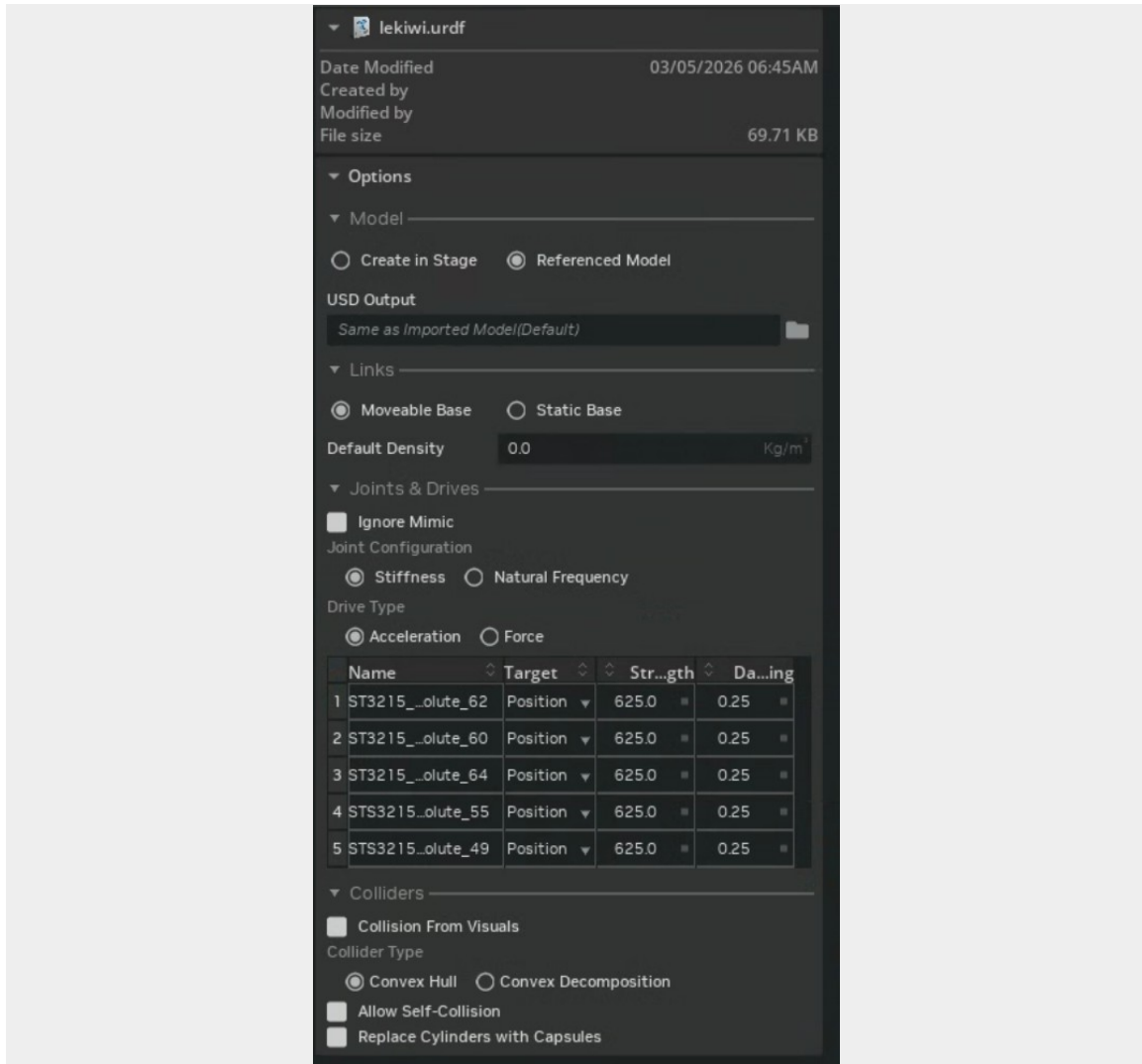
```
./isaac-sim.selector.sh
```

Lekiwi usd 설치

```
git clone https://github.com/kabilankb/lekiwi_isaacsim.git
cd lekiwi_isaacsim
```

3.1 Lekiwi 로봇 usd 로드

Isaac sim 을 실행한 뒤 File->Import->/home/yourname/lekiwi_isaacsim/urdf 경로를 클릭해 lekiwi.urdf 를 선택한다
이후 아래 사진과 같이 선택하여 import 한다.



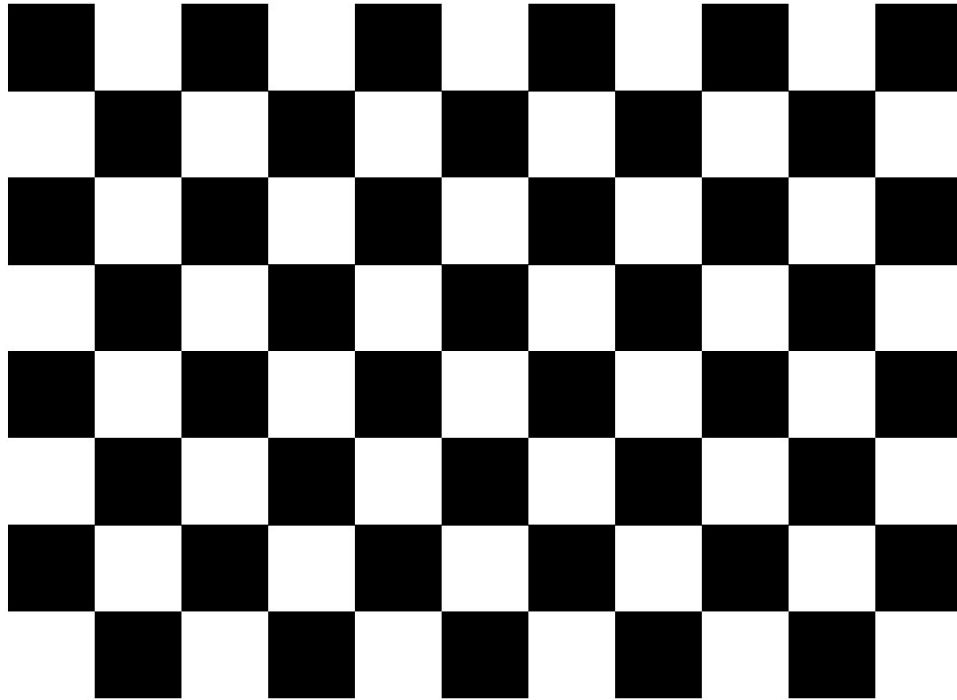
Lekiwi 로봇은 옴니휠 구조로 3 개의 바퀴가 120 도 간격으로 되어있어 바퀴마다 롤러가 자유관절로 설정되지 않으면 제대로된 전후진을 할 수 없다
 하지만 이 usd 같은 경우 바퀴의 롤러가 바퀴와 함께 하나의 mesh 로 설정되어 있어 휠은 Isaac sim 자체에서 제공하는 동일한 옴니휠 구조 로봇인 Kaya 로봇의 바퀴로 대체한다

첨부파일: lekiwi2.usd



3.2 카메라 적용

3.2.0 실물 로봇 카메라 파라미터 측정



www.calib.io | 8x11 | Checker Size: 15 mm.

체커보드를 활용해 실물 로봇에 부착된 base cam 과 wrist cam 의 내부/외부 파라미터를 측정한다

아래 코드의 base_cam 은 d455 용, wrist_cam 은 일반 usb 카메라 용으로 인자는 가로 / 세로 / 체커보드 한칸 크기 등이 있고 이때 8x11 이면 7x10 으로 입력해야 한다

```
python3 base_cam.py --do-chessboard-calib --pattern-cols 10 --pattern-rows 7 --square-size-m 0.015 --color-w 1280 --color-h 720 --color-fps 30 --min-samples 25 --show --out d455.json
```

```
python3 wrist_cam.py --device /dev/video2 --pattern-cols 10 --pattern-rows 7 --square-size-m 0.015 --tf-base-frame base_link --tf-camera-frame wristcam --show --out wrist_usb_cam.json
```

base_cam.py (전체 코드)

```
#!/usr/bin/env python3
# calib_dump_realsense_d455.py

import argparse
import json
import math
import os
import time
from typing import Any, Dict, List, Tuple

import numpy as np
```

```
import cv2
```

```
def quat_to_rpy(qx: float, qy: float, qz: float, qw: float) -> Tuple[float, float, float]:
```

```
    sinr_cosp = 2.0 * (qw * qx + qy * qz)
    cosr_cosp = 1.0 - 2.0 * (qx * qx + qy * qy)
    roll = math.atan2(sinr_cosp, cosr_cosp)
```

```
    sinp = 2.0 * (qw * qy - qz * qx)
    if abs(sinp) >= 1:
        pitch = math.copysign(math.pi / 2, sinp)
    else:
        pitch = math.asin(sinp)
```

```
    siny_cosp = 2.0 * (qw * qz + qx * qy)
    cosy_cosp = 1.0 - 2.0 * (qy * qy + qz * qz)
    yaw = math.atan2(siny_cosp, cosy_cosp)
    return roll, pitch, yaw
```

```
def compute_fov_deg(width: int, height: int, fx: float, fy: float) -> Tuple[float, float]:
```

```
    hfov = 2.0 * math.atan(width / (2.0 * fx))
    vfov = 2.0 * math.atan(height / (2.0 * fy))
    return math.degrees(hfov), math.degrees(vfov)
```

```
def write_ros_yaml(
```

```
    path: str,
    camera_name: str,
    width: int,
    height: int,
    K: np.ndarray,
    D: np.ndarray,
    distortion_model: str = "plumb_bob",
)
```

```
-> None:
```

```
    K_list = K.reshape(-1).tolist()
    D_list = D.reshape(-1).tolist()
    R_list = np.eye(3, dtype=float).reshape(-1).tolist()
    P = np.zeros((3, 4), dtype=float)
    P[0, 0] = K[0, 0]
    P[1, 1] = K[1, 1]
    P[0, 2] = K[0, 2]
    P[1, 2] = K[1, 2]
    P[2, 2] = 1.0
    P_list = P.reshape(-1).tolist()
```

```
def fmt_list(xs: List[float]) -> str:
```

```
    return "[" + ", ".join([f"{x:.12g}" for x in xs]) + "]"
```

```
yaml_text = (
```

```
    f"image_width: {width}\n"
    f"image_height: {height}\n"
    f"camera_name: {camera_name}\n"
    f"camera_matrix:\n"
    f"  rows: 3\n"
    f"  cols: 3\n"
    f"  data: {fmt_list(K_list)}\n"
    f"distortion_model: {distortion_model}\n"
    f"distortion_coefficients:\n"
    f"  rows: 1\n"
    f"  cols: {len(D_list)}\n"
    f"  data: {fmt_list(D_list)}\n"
    f"rectification_matrix:\n"
    f"  rows: 3\n"
```

```

        f" cols: 3\n"
        f" data: {fmt_list(R_list)}\n"
        f"projection_matrix:\n"
        f" rows: 3\n"
        f" cols: 4\n"
        f" data: {fmt_list(P_list)}\n"
    )

    with open(path, "w", encoding="utf-8") as f:
        f.write(yaml_text)

def try_ros2_tf(base_frame: str, camera_frame: str, timeout_s: float = 0.7) -> Dict[str, Any]:
    try:
        import rclpy # type: ignore
        import tf2_ros # type: ignore
        from rclpy.node import Node # type: ignore
        from rclpy.duration import Duration # type: ignore
    except Exception as e:
        return {"ok": False, "reason": f"ROS2 python deps not available: {e}"}

    class TFNode(Node):
        def __init__(self):
            super().__init__("tf_lookup_d455")
            self.buf = tf2_ros.Buffer()
            self.listener = tf2_ros.TransformListener(self.buf, self)

    rclpy.init(args=None)
    node = TFNode()

    t_start = time.time()
    tf_msg = None
    while time.time() - t_start < timeout_s:
        rclpy.spin_once(node, timeout_sec=0.1)
        try:
            tf_msg = node.buf.lookup_transform(
                base_frame, camera_frame, rclpy.time.Time(),
                timeout=Duration(seconds=0.2),
            )
            break
        except Exception:
            tf_msg = None

    if tf_msg is None:
        out = {"ok": False, "reason": f"TF not found: {base_frame} -> {camera_frame}"}
    else:
        t = tf_msg.transform.translation
        q = tf_msg.transform.rotation
        rpy = quat_to_rpy(q.x, q.y, q.z, q.w)
        out = {
            "ok": True,
            "base_frame": base_frame,
            "camera_frame": camera_frame,
            "translation_xyz_m": [float(t.x), float(t.y), float(t.z)],
            "quaternion_xyzw": [float(q.x), float(q.y), float(q.z), float(q.w)],
            "rpy_rad": [float(rpy[0]), float(rpy[1]), float(rpy[2])],
        }

    node.destroy_node()
    rclpy.shutdown()
    return out

def pick_realsense_device(rs, rs_serial: str) -> Tuple[Any, str]:

```



```

ctx = rs.context()
devices = ctx.query_devices()
if len(devices) == 0:
    raise RuntimeError("No RealSense device found")

if rs_serial:
    for d in devices:
        s = d.get_info(rs.camera_info.serial_number)
        if s == rs_serial:
            return d, s
    raise RuntimeError(f"RealSense serial '{rs_serial}' not found. Connected:
{[d.get_info(rs.camera_info.serial_number) for d in devices]}")
else:
    d0 = devices[0]
    s0 = d0.get_info(rs.camera_info.serial_number)
    return d0, s0

def rs_device_dump(
    color_w: int, color_h: int, color_fps: int,
    depth_w: int, depth_h: int, depth_fps: int,
    rs_serial: str,
) -> Dict[str, Any]:
    try:
        import pyrealsense2 as rs # type: ignore
    except Exception as e:
        return {"ok": False, "reason": f"pyrealsense2 not available: {e}"}

    try:
        dev, serial = pick_realsense_device(rs, rs_serial)
    except Exception as e:
        return {"ok": False, "reason": str(e)}

    name = dev.get_info(rs.camera_info.name) if dev.supports(rs.camera_info.name) else "unknown"
    fw = dev.get_info(rs.camera_info.firmware_version) if dev.supports(rs.camera_info.firmware_version)
else "unknown"
    usb = dev.get_info(rs.camera_info.usb_type_descriptor) if
dev.supports(rs.camera_info.usb_type_descriptor) else "unknown"

    depth_scale = None
    try:
        depth_scale = float(dev.first_depth_sensor().get_depth_scale())
    except Exception:
        pass

    pipe = rs.pipeline()
    cfg = rs.config()
    cfg.enable_device(serial)
    cfg.enable_stream(rs.stream.color, color_w, color_h, rs.format.rgb8, color_fps)
    cfg.enable_stream(rs.stream.depth, depth_w, depth_h, rs.format.z16, depth_fps)

    try:
        prof = pipe.start(cfg)
        color_stream = prof.get_stream(rs.stream.color).as_video_stream_profile()
        depth_stream = prof.get_stream(rs.stream.depth).as_video_stream_profile()

        c_intr = color_stream.get_intrinsics()
        d_intr = depth_stream.get_intrinsics()

        extr_d2c = depth_stream.get_extrinsics_to(color_stream)

    return {
        "ok": True,
        "device": {"name": name, "serial": serial, "firmware": fw, "usb": usb},

```

```

        "depth_scale_m_per_unit": depth_scale,
        "requested_streams": {
            "color": {"width": color_w, "height": color_h, "fps": color_fps, "format": "rgb8"},
            "depth": {"width": depth_w, "height": depth_h, "fps": depth_fps, "format": "z16"},
        },
        "intrinsics_from_device": {
            "color": {
                "width": int(c_intr.width),
                "height": int(c_intr.height),
                "fx": float(c_intr.fx),
                "fy": float(c_intr.fy),
                "ppx": float(c_intr.ppx),
                "ppy": float(c_intr.ppy),
                "model": str(c_intr.model),
                "coeffs": [float(x) for x in list(c_intr.coeffs)],
            },
            "depth": {
                "width": int(d_intr.width),
                "height": int(d_intr.height),
                "fx": float(d_intr.fx),
                "fy": float(d_intr.fy),
                "ppx": float(d_intr.ppx),
                "ppy": float(d_intr.ppy),
                "model": str(d_intr.model),
                "coeffs": [float(x) for x in list(d_intr.coeffs)],
            },
        },
        "extrinsics_from_device": {
            "depth_to_color": {
                "rotation_rowmajor_3x3": [float(x) for x in list(extr_d2c.rotation)],
                "translation_xyz_m": [float(x) for x in list(extr_d2c.translation)],
            }
        },
    }
except Exception as e:
    return {"ok": False, "reason": f"Failed to start pipeline or query intrinsics: {e}"}
finally:
    try:
        pipe.stop()
    except Exception:
        pass

def chessboard_calib_color_stream(
    color_w: int, color_h: int, color_fps: int,
    pattern_cols: int, pattern_rows: int, square_size_m: float,
    min_samples: int, max_samples: int, capture_interval_s: float,
    show: bool,
    rs_serial: str,
) -> Dict[str, Any]:
    try:
        import pyrealsense2 as rs # type: ignore
    except Exception as e:
        return {"ok": False, "reason": f"pyrealsense2 not available: {e}"}

    try:
        _, serial = pick_realsense_device(rs, rs_serial)
    except Exception as e:
        return {"ok": False, "reason": str(e)}

    pattern_size = (pattern_cols, pattern_rows)

    objp = np.zeros((pattern_rows * pattern_cols, 3), np.float32)
    objp[:, :2] = np.mgrid[0:pattern_cols, 0:pattern_rows].T.reshape(-1, 2)

```

```

objp *= float(square_size_m)

objpoints: List[np.ndarray] = []
imgpoints: List[np.ndarray] = []

pipe = rs.pipeline()
cfg = rs.config()
cfg.enable_device(serial)
cfg.enable_stream(rs.stream.color, color_w, color_h, rs.format.rgb8, color_fps)

prof = pipe.start(cfg)
last_cap_t = 0.0

print("\n[INFO] D455 color chessboard capture... (press q to finish & calibrate, r to reset)\n")

try:
    while True:
        frames = pipe.wait_for_frames(timeout_ms=2000)
        c = frames.get_color_frame()
        if not c:
            continue
        img = np.asanyarray(c.get_data()) # RGB
        img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
        gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
        h, w = gray.shape[:2]

        found = False
        corners = None
        if hasattr(cv2, "findChessboardCornersSB"):
            found, corners = cv2.findChessboardCornersSB(gray, pattern_size)

        if not found:
            flags = cv2.CALIB_CB_ADAPTIVE_THRESH | cv2.CALIB_CB_NORMALIZE_IMAGE
            found, corners = cv2.findChessboardCorners(gray, pattern_size, flags)

        vis = img_bgr.copy()
        status = f"samples={len(objpoints)}/{max_samples} found={found}"

        now = time.time()
        if found and corners is not None:
            criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-4)
            corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria) if corners.ndim == 3 else
corners
            cv2.drawChessboardCorners(vis, pattern_size, corners2, True)

            if (now - last_cap_t) >= capture_interval_s and len(objpoints) < max_samples:
                objpoints.append(objp.copy())
                imgpoints.append(corners2.reshape(-1, 2).copy())
                last_cap_t = now
                print(f"[CAPTURE] {len(objpoints):02d} (w={w}, h={h})")

        if show:
            cv2.putText(
                vis, status, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                (0, 255, 0) if found else (0, 0, 255), 2
            )
            cv2.imshow("d455_color_calib", vis)
            key = cv2.waitKey(1) & 0xFF
        else:
            key = cv2.waitKey(1) & 0xFF

        if key == ord("q"):
            break
        if key == ord("r"):

```

```

        objpoints.clear()
        imgpoints.clear()
        last_cap_t = 0.0
        print("[RESET] samples cleared")

    if len(objpoints) >= max_samples:
        print("[INFO] Reached max samples.")
        break

finally:
    try:
        pipe.stop()
    except Exception:
        pass
    try:
        cv2.destroyAllWindows()
    except Exception:
        pass

if len(objpoints) < min_samples:
    return {"ok": False, "reason": f"Not enough samples: got {len(objpoints)}, need >= {min_samples}"}

print(f"\n[INFO] Calibrating D455 color with {len(objpoints)} samples ...")
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    [op.reshape(-1, 3).astype(np.float32) for op in objpoints],
    [ip.reshape(-1, 1, 2).astype(np.float32) for ip in imgpoints],
    (w, h),
    None,
    None,
)

total_err = 0.0
total_pts = 0
for i in range(len(objpoints)):
    imgpts2, _ = cv2.projectPoints(objpoints[i], rvecs[i], tvecs[i], K, dist)
    err = cv2.norm(imgpoints[i].reshape(-1, 1, 2), imgpts2, cv2.NORM_L2)
    total_err += err * err
    total_pts += len(imgpoints[i])
rmse = math.sqrt(total_err / max(total_pts, 1))

fx, fy = float(K[0, 0]), float(K[1, 1])
cx, cy = float(K[0, 2]), float(K[1, 2])
hfov_deg, vfov_deg = compute_fov_deg(w, h, fx, fy)

return {
    "ok": True,
    "calibration_method": {
        "type": "chessboard_opencv_calibrateCamera",
        "pattern_inner_corners_cols": pattern_cols,
        "pattern_inner_corners_rows": pattern_rows,
        "square_size_m": float(square_size_m),
        "samples_used": len(objpoints),
    },
    "image": {"width": int(w), "height": int(h)},
    "intrinsics": {
        "distortion_model": "plumb_bob",
        "K_rowmajor_3x3": K.reshape(-1).tolist(),
        "D": dist.reshape(-1).tolist(),
        "fx_fy_cx_cy": [fx, fy, cx, cy],
        "hfov_deg": hfov_deg,
        "vfov_deg": vfov_deg,
        "opencv_ret": float(ret),
        "reprojection_rmse_px": float(rmse),
    },
}

```

```

}

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default="realsense_d455.json", help="Output JSON path")
    ap.add_argument("--camera-name", default="base_realsense_d455", help="Camera name in outputs")

    ap.add_argument("--rs-serial", default="", help="RealSense serial to select (empty=first device)")

    ap.add_argument("--color-w", type=int, default=640)
    ap.add_argument("--color-h", type=int, default=480)
    ap.add_argument("--color-fps", type=int, default=30)
    ap.add_argument("--depth-w", type=int, default=640)
    ap.add_argument("--depth-h", type=int, default=480)
    ap.add_argument("--depth-fps", type=int, default=30)

    ap.add_argument("--do-chessboard-calib", action="store_true", help="run chessboard calibration on color stream")
    ap.add_argument("--pattern-cols", type=int, default=10)
    ap.add_argument("--pattern-rows", type=int, default=7)
    ap.add_argument("--square-size-m", type=float, default=0.015)
    ap.add_argument("--min-samples", type=int, default=25)
    ap.add_argument("--max-samples", type=int, default=60)
    ap.add_argument("--capture-interval-s", type=float, default=0.6)
    ap.add_argument("--show", action="store_true")

    ap.add_argument("--tf-base-frame", default="", help="if set, try TF lookup base->camera")
    ap.add_argument("--tf-camera-frame", default="", help="camera frame name in TF (e.g., base_rgb, camera_link, etc.)")

    args = ap.parse_args()

    device_info = rs_device_dump(
        color_w=args.color_w, color_h=args.color_h, color_fps=args.color_fps,
        depth_w=args.depth_w, depth_h=args.depth_h, depth_fps=args.depth_fps,
        rs_serial=args.rs_serial,
    )

    chess_calib = None
    if args.do_chessboard_calib:
        chess_calib = chessboard_calib_color_stream(
            color_w=args.color_w, color_h=args.color_h, color_fps=args.color_fps,
            pattern_cols=args.pattern_cols, pattern_rows=args.pattern_rows,
            square_size_m=args.square_size_m,
            min_samples=args.min_samples, max_samples=args.max_samples,
            capture_interval_s=args.capture_interval_s, show=args.show,
            rs_serial=args.rs_serial,
        )

    tf_dump = None
    if args.tf_base_frame and args.tf_camera_frame:
        tf_dump = try_ros2_tf(args.tf_base_frame, args.tf_camera_frame)

    out: Dict[str, Any] = {
        "camera_role": "base_realsense_d455",
        "camera_name": args.camera_name,
        "timestamp_unix": time.time(),
        "device_dump": device_info,
        "chessboard_color_calibration": chess_calib,
        "tf_base_to_camera": tf_dump,
        "isaac_sim_notes": [
            "D455 는 보통 'device_dump(intrinsics_from_device)' 값을 그대로 쓰는 게 가장 안정적입니다.",

```

"체커보드로 구한 값(chessboard_color_calibration)은 설치/포커스/렌즈 상태를 반영하지만 샘플 품질에 따라 달라질 수 있습니다.",

"Isaac Sim 카메라 설정은 FOV(hfov/vfov) 또는 fx/fy 기반으로 맞추면 됩니다.",

"로봇 기준 pose는 TF가 있으면 그대로 쓰고, 없으면 URDF/CAD 기준으로 xyz/rpy를 수동 입력해야 합니다.",

```
    ],
}

out_path = os.path.abspath(args.out)
os.makedirs(os.path.dirname(out_path), exist_ok=True)
with open(out_path, "w", encoding="utf-8") as f:
    json.dump(out, f, indent=2, ensure_ascii=False)

yaml_path = os.path.splitext(out_path)[0] + "_ros.yaml"

if chess_calib and chess_calib.get("ok", False):
    K = np.array(chess_calib["intrinsic"]["K_rowmajor_3x3"], dtype=float).reshape(3, 3)
    D = np.array(chess_calib["intrinsic"]["D"], dtype=float).reshape(-1, 1)
    width = int(chess_calib["image"]["width"])
    height = int(chess_calib["image"]["height"])
    write_ros_yaml(yaml_path, args.camera_name + "_color", width, height, K, D, "plumb_bob")
else:
    if device_info.get("ok", False):
        c = device_info["intrinsic_from_device"]["color"]
        width = int(c["width"])
        height = int(c["height"])
        fx, fy, ppx, ppy = float(c["fx"]), float(c["fy"]), float(c["ppx"]), float(c["ppy"])
        K = np.array([[fx, 0.0, ppx],
                      [0.0, fy, ppy],
                      [0.0, 0.0, 1.0]], dtype=float)
        D = np.array(c["coeffs"], dtype=float).reshape(-1, 1)
        write_ros_yaml(yaml_path, args.camera_name + "_color_device", width, height, K, D, "plumb_bob")
    else:
        yaml_path = ""

print("\n===== RESULT =====")
print(json.dumps(out, indent=2, ensure_ascii=False))
print(f"\nSaved JSON : {out_path}")
if yaml_path:
    print(f"Saved YAML : {yaml_path}")
else:
    print("YAML not saved (no valid intrinsics available).")

if __name__ == "__main__":
    main()
```

base_result (측정 결과)

```
===== RESULT =====
{
  "camera_role": "base_realsense_d455",
  "camera_name": "base_realsense_d455",
  "timestamp_unix": 1768883668.7049365,
  "device_dump": {
    "ok": true,
    "device": {
      "name": "Intel RealSense D455",
      "serial": "341522300432",
      "firmware": "5.17.0.10",
      "usb": "3.2"
    },
    "depth_scale_m_per_unit": 0.0010000000474974513,
    "requested_streams": {
      "color": { "width": 1280, "height": 720, "fps": 30, "format": "rgb8" },
```

```

    "depth": {"width": 640, "height": 480, "fps": 30, "format": "z16"}
  },
  "intrinsic_from_device": {
    "color": {
      "width": 1280, "height": 720,
      "fx": 648.3814697265625, "fy": 647.343017578125,
      "ppx": 640.1439819335938, "ppy": 376.93280029296875,
      "model": "distortion.inverse_brown_conrady",
      "coeffs": [-0.0543576255440712, 0.062036074697971344, 0.00019799006986431777,
0.0013188293669372797, -0.02017592452466488]
    },
    "depth": {
      "width": 640, "height": 480,
      "fx": 391.0513916015625, "fy": 391.0513916015625,
      "ppx": 321.46832275390625, "ppy": 241.74258422851562,
      "model": "distortion.brown_conrady",
      "coeffs": [0.0, 0.0, 0.0, 0.0, 0.0]
    }
  },
  "extrinsic_from_device": {
    "depth_to_color": {
      "rotation_rowmajor_3x3": [0.9999931454658508, 0.0004615642537828535, -
0.0036799858789891005, -0.0004657019453588873, 0.9999992847442627, -0.0011235992424190044,
0.0036794645711779594, 0.0011253051925450563, 0.9999926090240479],
      "translation_xyz_m": [-0.05909388139843941, 5.698289896827191e-05, 0.0005283755017444491]
    }
  },
  "chessboard_color_calibration": {
    "ok": true,
    "calibration_method": {
      "type": "chessboard_opencv_calibrateCamera",
      "pattern_inner_corners_cols": 10, "pattern_inner_corners_rows": 7,
      "square_size_m": 0.015, "samples_used": 60
    },
    "image": {"width": 1280, "height": 720},
    "intrinsic": {
      "distortion_model": "plumb_bob",
      "K_rowmajor_3x3": [606.5395528769419, 0.0, 644.9236816573458, 0.0, 606.0112025477338,
379.5410188544695, 0.0, 0.0, 1.0],
      "D": [-0.12440243198578378, 0.09083581567322827, 0.003687195814377516,
0.0011456777153378186, -0.21215993965300448],
      "fx_fy_cx_cy": [606.5395528769419, 606.0112025477338, 644.9236816573458,
379.5410188544695],
      "hfov_deg": 93.07520470280036, "vfov_deg": 61.424725280423196,
      "opencv_ret": 0.31166625807848247, "reprojection_rmse_px": 0.31166595505345884
    }
  },
  "tf_base_to_camera": null,
  "isaac_sim_notes": [
    "D455 는 보통 'device_dump(intrinsic_from_device)' 값을 그대로 쓰는 게 가장 안정적입니다.",
    "체커보드로 구한 값(chessboard_color_calibration)은 설치/포커스/렌즈 상태를 반영하지만 샘플 품질에 따라 달라질 수 있습니다.",
    "Isaac Sim 카메라 설정은 FOV(hfov/vfov) 또는 fx/fy 기반으로 맞추면 됩니다.",
    "로봇 기준 pose 는 TF 가 있으면 그대로 쓰고, 없으면 URDF/CAD 기준으로 xyz/rpy 를 수동 입력해야 합니다."
  ]
}

```

Saved JSON : /home/yubin/d455.json

Saved YAML : /home/yubin/d455_ros.yaml

wrist_cam.py (전체 코드)

```
#!/usr/bin/env python3
```

```

# calib_dump_wrist_usb_cam.py
#
# USB wrist camera 를 체크보드로 캘리브레이션하여
# - intrinsics(K), distortion(D), reprojection error
# - 해상도/추정 FPS
# - (선택) ROS2 TF(base_frame -> camera_frame) pose
# 를 JSON + ROS camera_calibration YAML 로 저장합니다.
#
# 사용 예:
# 1) 기본(장치 /dev/video0, 코너 9x6, 한칸 0.025m):
#   python3 calib_dump_wrist_usb_cam.py --device /dev/video0 --out wrist_usb_cam.json
#
# 2) 코너/칸크기 지정 + TF 도 덤프(ROS2 환경):
#   python3 calib_dump_wrist_usb_cam.py \
#     --device /dev/video0 \
#     --pattern-cols 9 --pattern-rows 6 --square-size-m 0.025 \
#     --tf-base-frame base_link --tf-camera-frame wristcam \
#     --out wrist_usb_cam.json
#
# 조작:
# - 화면에서 체크보드 인식되면 자동으로 샘플 수집(기본 0.6 초 간격)
# - q: 종료하고 캘리브레이션 실행
# - r: 지금까지 수집한 샘플 리셋
#
# 출력:
# - <out>.json
# - <out_basename>_ros.yaml

```

```

import argparse
import json
import math
import os
import time
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import cv2

```

```

def quat_to_rpy(qx: float, qy: float, qz: float, qw: float) -> Tuple[float, float, float]:
    # roll
    sinr_cosp = 2.0 * (qw * qx + qy * qz)
    cosr_cosp = 1.0 - 2.0 * (qx * qx + qy * qy)
    roll = math.atan2(sinr_cosp, cosr_cosp)
    # pitch
    sinp = 2.0 * (qw * qy - qz * qx)
    if abs(sinp) >= 1:
        pitch = math.copysign(math.pi / 2, sinp)
    else:
        pitch = math.asin(sinp)
    # yaw
    siny_cosp = 2.0 * (qw * qz + qx * qy)
    cosy_cosp = 1.0 - 2.0 * (qy * qy + qz * qz)
    yaw = math.atan2(siny_cosp, cosy_cosp)
    return roll, pitch, yaw

```

```

def write_ros_yaml(
    path: str,
    camera_name: str,
    width: int,
    height: int,
    K: np.ndarray,
    D: np.ndarray,

```



```

    distortion_model: str = "plumb_bob",
) -> None:
    # ROS camera_calibration 형식(수동 작성)
    K_list = K.reshape(-1).tolist()
    D_list = D.reshape(-1).tolist()
    R_list = np.eye(3, dtype=float).reshape(-1).tolist()
    P = np.zeros((3, 4), dtype=float)
    P[0, 0] = K[0, 0]
    P[1, 1] = K[1, 1]
    P[0, 2] = K[0, 2]
    P[1, 2] = K[1, 2]
    P[2, 2] = 1.0
    P_list = P.reshape(-1).tolist()

def fmt_list(xs: List[float]) -> str:
    return "[" + ", ".join([f"{x:.12g}" for x in xs]) + "]"

yaml_text = (
    f"image_width: {width}\n"
    f"image_height: {height}\n"
    f"camera_name: {camera_name}\n"
    f"camera_matrix:\n"
    f"  rows: 3\n"
    f"  cols: 3\n"
    f"  data: {fmt_list(K_list)}\n"
    f"distortion_model: {distortion_model}\n"
    f"distortion_coefficients:\n"
    f"  rows: 1\n"
    f"  cols: {len(D_list)}\n"
    f"  data: {fmt_list(D_list)}\n"
    f"rectification_matrix:\n"
    f"  rows: 3\n"
    f"  cols: 3\n"
    f"  data: {fmt_list(R_list)}\n"
    f"projection_matrix:\n"
    f"  rows: 3\n"
    f"  cols: 4\n"
    f"  data: {fmt_list(P_list)}\n"
)

with open(path, "w", encoding="utf-8") as f:
    f.write(yaml_text)

def compute_fov_deg(width: int, height: int, fx: float, fy: float) -> Tuple[float, float]:
    # FOVx = 2*atan(w/(2*fx)), FOVy = 2*atan(h/(2*fy))
    hfov = 2.0 * math.atan(width / (2.0 * fx))
    vfov = 2.0 * math.atan(height / (2.0 * fy))
    return math.degrees(hfov), math.degrees(vfov)

def try_ros2_tf(base_frame: str, camera_frame: str, timeout_s: float = 0.7) -> Dict[str, Any]:
    """
    ROS2 TF(base_frame -> camera_frame) 한 번 lookup.
    ROS2 환경이 아니면 ok:false 반환.
    """
    try:
        import rclpy # type: ignore
        import tf2_ros # type: ignore
        from rclpy.node import Node # type: ignore
        from rclpy.duration import Duration # type: ignore
    except Exception as e:
        return {"ok": False, "reason": f"ROS2 python deps not available: {e}"}

```

```

class TFNode(Node):
    def __init__(self):
        super().__init__("tf_lookup_wrist_cam")
        self.buf = tf2_ros.Buffer()
        self.listener = tf2_ros.TransformListener(self.buf, self)

rclpy.init(args=None)
node = TFNode()

t_start = time.time()
tf_msg = None
while time.time() - t_start < timeout_s:
    rclpy.spin_once(node, timeout_sec=0.1)
    try:
        tf_msg = node.buf.lookup_transform(
            base_frame, camera_frame, rclpy.time.Time(),
            timeout=Duration(seconds=0.2),
        )
        break
    except Exception:
        tf_msg = None

out: Dict[str, Any]
if tf_msg is None:
    out = {"ok": False, "reason": f"TF not found: {base_frame} -> {camera_frame}"}
else:
    t = tf_msg.transform.translation
    q = tf_msg.transform.rotation
    rpy = quat_to_rpy(q.x, q.y, q.z, q.w)
    out = {
        "ok": True,
        "base_frame": base_frame,
        "camera_frame": camera_frame,
        "translation_xyz_m": [float(t.x), float(t.y), float(t.z)],
        "quaternion_xyzw": [float(q.x), float(q.y), float(q.z), float(q.w)],
        "rpy_rad": [float(rpy[0]), float(rpy[1]), float(rpy[2])],
    }

node.destroy_node()
rclpy.shutdown()
return out

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--device", default="/dev/video2", help="USB camera device (e.g., /dev/video0)")
    ap.add_argument("--out", default="wrist_usb_cam.json", help="Output JSON path")
    ap.add_argument("--camera-name", default="wrist_usb_cam", help="Camera name in outputs")

    # chessboard pattern (INNER corners)
    ap.add_argument("--pattern-cols", type=int, default=9, help="inner corners per row (columns)")
    ap.add_argument("--pattern-rows", type=int, default=6, help="inner corners per column (rows)")
    ap.add_argument("--square-size-m", type=float, default=0.025, help="square size in meters")

    # capture control
    ap.add_argument("--min-samples", type=int, default=25, help="minimum samples required to calibrate")
    ap.add_argument("--max-samples", type=int, default=60, help="maximum samples to collect")
    ap.add_argument("--capture-interval-s", type=float, default=0.6, help="auto-capture interval when board detected")
    ap.add_argument("--show", action="store_true", help="show preview window")

    # (optional) TF dump
    ap.add_argument("--tf-base-frame", default="", help="if set, try TF lookup base->camera")

```

```

ap.add_argument("--tf-camera-frame", default="", help="camera frame name in TF")

args = ap.parse_args()

pattern_size = (args.pattern_cols, args.pattern_rows)

# object points grid
objp = np.zeros((args.pattern_rows * args.pattern_cols, 3), np.float32)
objp[:, :2] = np.mgrid[0:args.pattern_cols, 0:args.pattern_rows].T.reshape(-1, 2)
objp *= float(args.square_size_m)

objpoints: List[np.ndarray] = []
imgpoints: List[np.ndarray] = []

cap = cv2.VideoCapture(args.device)
if not cap.isOpened():
    raise SystemExit(f"Failed to open camera device: {args.device}")

last_cap_t = 0.0
frame_count = 0
fps_t0 = time.time()

print("\n[INFO] Looking for chessboard... (press q to finish & calibrate, r to reset)\n")

while True:
    ok, frame = cap.read()
    if not ok or frame is None:
        print("[WARN] frame read failed")
        time.sleep(0.05)
        continue

    frame_count += 1
    now = time.time()
    if now - fps_t0 < 5.0:
        pass

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape[:2]

    flags = cv2.CALIB_CB_ADAPTIVE_THRESH | cv2.CALIB_CB_NORMALIZE_IMAGE
    found, corners = cv2.findChessboardCorners(gray, pattern_size, flags)

    vis = frame.copy()
    status_text = f"samples={len(objpoints)}/{args.max_samples} found={found}"

    if found and corners is not None:
        # refine corners
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-4)
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
        cv2.drawChessboardCorners(vis, pattern_size, corners2, found)

        # auto-capture
        if (now - last_cap_t) >= args.capture_interval_s and len(objpoints) < args.max_samples:
            objpoints.append(objp.copy())
            imgpoints.append(corners2.reshape(-1, 2).copy())
            last_cap_t = now
            print(f"[CAPTURE] {len(objpoints):02d} (w={w}, h={h})")

    if args.show:
        cv2.putText(vis, status_text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0) if found else
(0, 0, 255), 2)
        cv2.imshow("wrist_usb_cam_calib", vis)
        key = cv2.waitKey(1) & 0xFF
    else:

```

```

        key = cv2.waitKey(1) & 0xFF # still allows key interrupts on some systems

    if key == ord("q"):
        break
    if key == ord("r"):
        objpoints.clear()
        imgpoints.clear()
        last_cap_t = 0.0
        print("[RESET] samples cleared")

    if len(objpoints) >= args.max_samples:
        print("[INFO] Reached max samples.")
        break

cap.release()
try:
    cv2.destroyAllWindows()
except Exception:
    pass

if len(objpoints) < args.min_samples:
    raise SystemExit(f"Not enough samples to calibrate: got {len(objpoints)}, need >= {args.min_samples}")

# Calibrate
print(f"\n[INFO] Calibrating with {len(objpoints)} samples ...")
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    [op.reshape(-1, 3).astype(np.float32) for op in objpoints],
    [ip.reshape(-1, 1, 2).astype(np.float32) for ip in imgpoints],
    (w, h),
    None,
    None,
)

# Reprojection RMSE
total_err = 0.0
total_pts = 0
for i in range(len(objpoints)):
    imgpts2, _ = cv2.projectPoints(objpoints[i], rvecs[i], tvecs[i], K, dist)
    err = cv2.norm(imgpoints[i].reshape(-1, 1, 2), imgpts2, cv2.NORM_L2)
    total_err += err * err
    total_pts += len(imgpoints[i])
rmse = math.sqrt(total_err / max(total_pts, 1))

fx, fy = float(K[0, 0]), float(K[1, 1])
cx, cy = float(K[0, 2]), float(K[1, 2])
hfov_deg, vfov_deg = compute_fov_deg(w, h, fx, fy)

# TF (optional)
tf_dump = None
if args.tf_base_frame and args.tf_camera_frame:
    tf_dump = try_ros2_tf(args.tf_base_frame, args.tf_camera_frame)

out: Dict[str, Any] = {
    "camera_role": "wrist_usb_cam",
    "camera_name": args.camera_name,
    "timestamp_unix": time.time(),
    "calibration_method": {
        "type": "chessboard_opencv_calibrateCamera",
        "pattern_inner_corners_cols": args.pattern_cols,
        "pattern_inner_corners_rows": args.pattern_rows,
        "square_size_m": float(args.square_size_m),
        "samples_used": len(objpoints),
    },
}

```

```

"image": {"width": int(w), "height": int(h)},
"intrinsics": {
    "distortion_model": "plumb_bob",
    "K_rowmajor_3x3": K.reshape(-1).tolist(),
    "D": dist.reshape(-1).tolist(),
    "fx_fy_cx_cy": [fx, fy, cx, cy],
    "hfov_deg": hfov_deg,
    "vfov_deg": vfov_deg,
    "opencv_ret": float(ret),
    "reprojection_rmse_px": float(rmse),
},
"tf_base_to_camera": tf_dump, # ok:false 일 수 있음
"isaac_sim_notes": [
    "Isaac Sim 카메라를 설정할 때 FOV(hfov/vfov) 또는 fx/fy 기반으로 맞추면 됩니다.",
    "왜곡(distortion)은 Isaac Sim 기본 pinhole 에선 직접 반영이 어려워 보통 '왜곡 보정된 이미지'를 쓰거나 도메인
    랜덤화로 흡수합니다.",
],
}

# Save JSON
out_path = os.path.abspath(args.out)
os.makedirs(os.path.dirname(out_path), exist_ok=True)
with open(out_path, "w", encoding="utf-8") as f:
    json.dump(out, f, indent=2, ensure_ascii=False)

# Save ROS YAML
yaml_path = os.path.splitext(out_path)[0] + "_ros.yaml"
write_ros_yaml(
    yaml_path,
    camera_name=args.camera_name,
    width=int(w),
    height=int(h),
    K=K.astype(float),
    D=dist.astype(float),
    distortion_model="plumb_bob",
)

print("\n===== RESULT =====")
print(json.dumps(out, indent=2, ensure_ascii=False))
print(f"\nSaved JSON : {out_path}")
print(f"\nSaved YAML : {yaml_path}")

if __name__ == "__main__":
    main()

```

wrist_result (측정 결과)

```

===== RESULT =====
{
  "camera_role": "wrist_usb_cam",
  "camera_name": "wrist_usb_cam",
  "timestamp_unix": 1768881366.1593406,
  "calibration_method": {
    "type": "chessboard_opencv_calibrateCamera",
    "pattern_inner_corners_cols": 10, "pattern_inner_corners_rows": 7,
    "square_size_m": 0.015, "samples_used": 60
  },
  "image": {"width": 640, "height": 480},
  "intrinsics": {
    "distortion_model": "plumb_bob",
    "K_rowmajor_3x3": [462.3983737919788, 0.0, 330.0634470177232, 0.0, 462.70399196137635,
267.1746903051712, 0.0, 0.0, 1.0],

```

```

"D": [-0.45191290130653683, 0.2781355512329491, -0.0015105871972495685,
0.0010508009075621148, -0.09480361956785176],
"fx_fy_cx_cy": [462.3983737919788, 462.70399196137635, 330.0634470177232,
267.1746903051712],
"hfov_deg": 69.36987452500811, "vfov_deg": 54.83064933502291,
"opencv_ret": 0.6228917407889041, "reprojection_rmse_px": 0.622891738478654
},
"tf_base_to_camera": {"ok": false, "reason": "TF not found: base_link -> wristcam"},
"isaac_sim_notes": [
  "Isaac Sim 카메라를 설정할 때 FOV(hfov/vfov) 또는 fx/fy 기반으로 맞추면 됩니다.",
  "왜곡(distortion)은 Isaac Sim 기본 pinhole 에선 직접 반영이 어려워 보통 '왜곡 보정된 이미지'를 쓰거나 도메인 랜덤화로
  흡수합니다."
]
}

```

Saved JSON : /home/yubin/wrist_usb_cam.json
 Saved YAML : /home/yubin/wrist_usb_cam_ros.yaml

3.2.1 Isaac sim 에 카메라 추가

basecam 에 사용하는 intel realsense d455 는 isaac sim 에서 제공되기 때문에 create->sensors->camera and depth sensors->intel->intel realsense d455 를 선택해 스폰하고 실제 위치로 이동시킨 뒤 Lekiwi 로봇의 articulation root 인 base_plate_layer1_v5 아래에 넣어 로봇에 부착시킨다

경로 : /World/LeKiwi/base_plate_layer1_v5/Realsense

wristcam 에 사용하는 일반 usb cam 은 create->camera 로 스폰한 뒤 수동으로 카메라 위치로 이동시키고 마찬가지로 wrist_roll_08c_v1 아래의 카메라 mesh 안에 넣어 로봇에 부착시킨다

경로 : /World/LeKiwi/Wrist_Roll_08c_v1/visuals/mesh_002_3/wrist_camera

3.2.2 실물 로봇의 카메라 파라미터 Isaac sim 에 적용

위에서 측정한 실물 카메라의 파라미터를 Isaac sim 의 script editor 를 통해 isaac sim 에 추가한 카메라에 적용한다.

아래의 코드 속 camera prim path, 측정값, 카메라 해상도 등을 변경하여 사용한다

calibration_script_editor_ver (전체 코드)

```

from pxr import Usd, UsdGeom, Gf
import omni.usd
import math

stage = omni.usd.get_context().get_stage()

# === Camera prim paths ===
CAM_WRIST = "/World/LeKiwi/Wrist_Roll_08c_v1/visuals/mesh_002_3/wrist_camera"
CAM_D455_COLOR =
"/World/LeKiwi/base_plate_layer1_v5/Realsense/RSD455/Camera_OmniVision_OV9782_Color"
CAM_D455_DEPTH = "/World/LeKiwi/base_plate_layer1_v5/Realsense/RSD455/Camera_Pseudo_Depth"

def fov_deg(w, h, fx, fy):
    hfov = 2.0 * math.atan(w / (2.0 * fx))
    vfov = 2.0 * math.atan(h / (2.0 * fy))
    return math.degrees(hfov), math.degrees(vfov)

def set_usd_camera_intrinsics(cam_path: str,
                              width: int, height: int,
                              fx: float, fy: float, cx: float, cy: float,
                              focal_mm: float = 6.0,
                              near: float = 0.01, far: float = 1000.0):
    """
    UsdGeom.Camera 는 (focalLength, aperture, apertureOffset)로 pinhole 을 표현.
    - pixel_size_x = focal_mm / fx
    """

```

```

- pixel_size_y = focal_mm / fy
- hap = pixel_size_x * width
- vap = pixel_size_y * height
- offset:
    ox = (cx - width/2) * pixel_size_x
    oy = (height/2 - cy) * pixel_size_y # y-down -> y-up 보정
"""

prim = stage.GetPrimAtPath(cam_path)
if not prim or not prim.IsValid():
    print(f"[FAIL] prim not found: {cam_path}")
    return False
if not prim.IsA(UsdGeom.Camera):
    print(f"[FAIL] prim is not UsdGeom.Camera: {cam_path} (type={prim.GetTypeName()})")
    return False

cam = UsdGeom.Camera(prim)

sx = focal_mm / float(fx)
sy = focal_mm / float(fy)

hap = sx * float(width)
vap = sy * float(height)

ox = (float(cx) - float(width) * 0.5) * sx
oy = (float(height) * 0.5 - float(cy)) * sy

cam.GetProjectionAttr().Set(UsdGeom.Tokens.perspective)
cam.GetFocalLengthAttr().Set(float(focal_mm))
cam.GetHorizontalApertureAttr().Set(float(hap))
cam.GetVerticalApertureAttr().Set(float(vap))
cam.GetHorizontalApertureOffsetAttr().Set(float(ox))
cam.GetVerticalApertureOffsetAttr().Set(float(oy))
cam.GetClippingRangeAttr().Set(Gf.Vec2f(float(near), float(far)))

hf, vf = fov_deg(width, height, fx, fy)
print(f"[OK] {cam_path}")
print(f"    res={width}x{height}")
print(f"    fx={fx:.6f} fy={fy:.6f} cx={cx:.6f} cy={cy:.6f}")
print(f"    focal_mm={focal_mm:.3f} hap_mm={hap:.6f} vap_mm={vap:.6f}")
print(f"    offset_mm=({ox:.6f}, {oy:.6f})")
print(f"    hfov≈{hf:.3f} deg vfov≈{vf:.3f} deg")
return True

# wrist USB cam (640x480)
wrist = dict(width=640, height=480,
             fx=462.3983737919788, fy=462.70399196137635,
             cx=330.0634470177232, cy=267.1746903051712)

# D455 color device intrinsics (1280x720)
d455_color = dict(width=1280, height=720,
                  fx=648.3814697265625, fy=647.343017578125,
                  cx=640.1439819335938, cy=376.93280029296875)

# D455 depth device intrinsics (640x480)
d455_depth = dict(width=640, height=480,
                  fx=391.0513916015625, fy=391.0513916015625,
                  cx=321.46832275390625, cy=241.74258422851562)

print("=== APPLY ===")
set_usd_camera_intrinsics(CAM_WRIST, **wrist)
set_usd_camera_intrinsics(CAM_D455_COLOR, **d455_color)
set_usd_camera_intrinsics(CAM_D455_DEPTH, **d455_depth)
print("[DONE]")

```

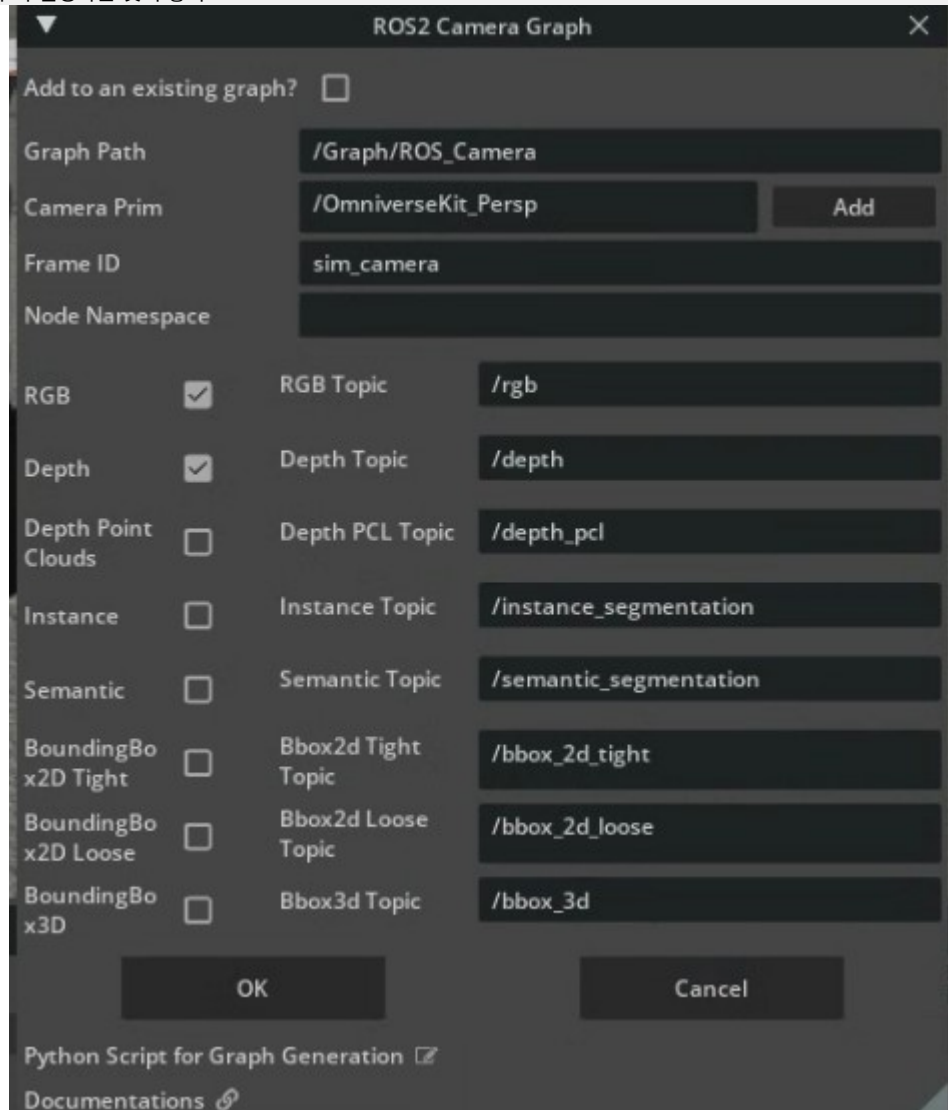
3.2.3 카메라 이미지 퍼블리시

이러한 카메라 이미지를 수집하기 위해 ros2 를 통해 퍼블리시하도록 진행한다

Tools->Ros2 OmniGraphs->camera

graph path 에서 옴니그래프의 이름을 수정하고 camera prim 에 add 를 통해 위에서 추가한 카메라를 선택한다. 이후 frameid 를 수정해 카메라의 이름을 변경하고 RGB, Depth 체크를 통해 퍼블리시 할 항목과 토픽명을 결정한 뒤 ok 를 누른다.

다만 D455 의 경우 센서 내부에 RGB 와 Depth 두 항목으로 나누어져 있어 한번에 두개를 모두 선택하면 추후 이미지 토픽을 확인했을 때 둘 다 제대로 나오지 않아 Color 와 depth 두개로 나누어 설정하는 것이 좋다



4. 원격조종

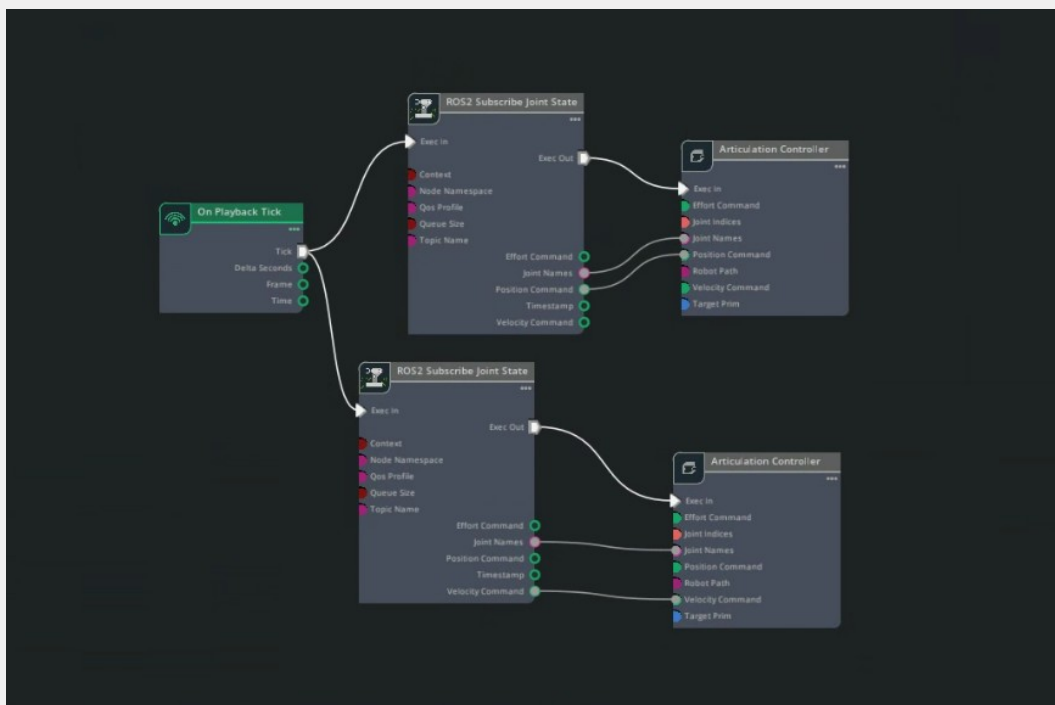
4.0 구성 설명

원격조종은 노트북에서 리더암/키보드로 명령을 내리면 Isaac sim 이 설치되어 있는 환경에서 이 명령을 받아 ros2 로 퍼블리시 하는 구조로 진행된다

송신기는 arm joint + base(vx,vy,wz)를 TCP JSON 으로 보내고,수신기는 이를 /leader_joint_states, /wheel_cmds 등 으로 publish 한다

4.1 Isaac sim action graph 생성

Action graph 는 Isaac sim 에서 제공하는 편리하게 노드 구조를 설정할 수 있는 툴로 tools->graph editors->action graph 후 열리는 창에서 new action graph 순으로 진행해 action graph 를 생성할 수 있다



위 그림과 같이 action graph 를 세팅하는데 이때 로봇팔은 position 제어, 휠은 velocity 제어이므로 이를 유의해서 연결한다.

Ros2 Subscriber 의 Property 에서 Inputs 항목에 topicName 을 실제로 서브스크라이브 받을 토픽 명으로 수정해야 한다.

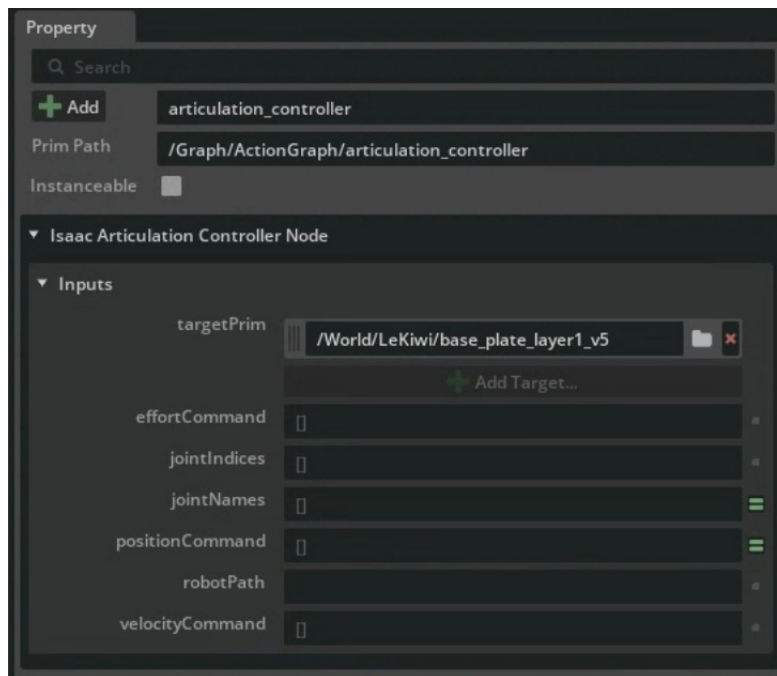
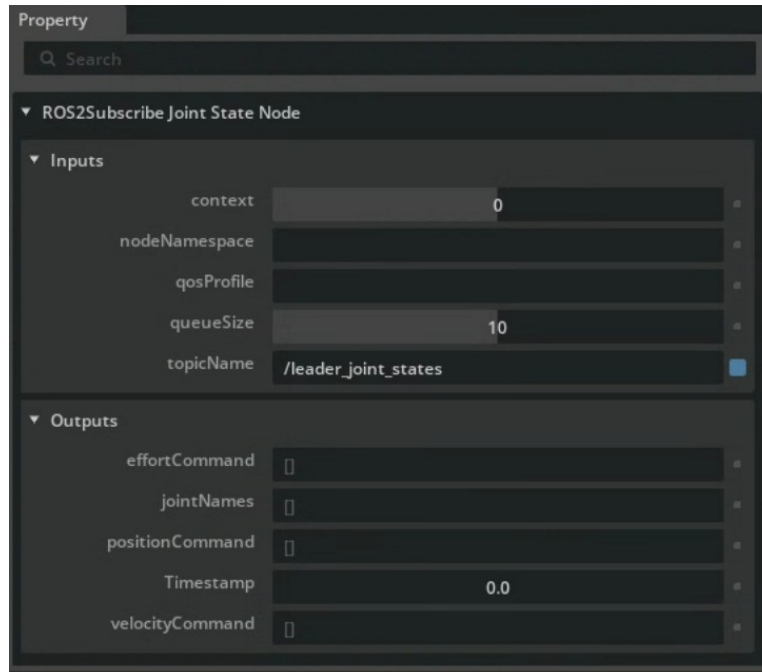
또 Articulation Controller 의 inputs 항목에 target prim 은 로봇의 실제 articulation root 를 찾아 지정해야 한다 만약 경로를 모른다면 window->script editor 에서

```
import omni.usd
from pxr import UsdPhysics

stage = omni.usd.get_context().get_stage()

for prim in stage.Traverse():
    if prim.HasAPI(UsdPhysics.ArticulationRootAPI):
        print(f"Articulation Root 경로: {prim.GetPath()}")
```

를 통해 찾을 수 있다



4.2 수신기

수신기의 코드는 기존 lerobot 플랫폼에서 제공하는 robots/lekiwi/lekiwi_host.py 에서 통신 방식을 TCP Socket 으로 변환하고 송신기에서 보낸 값을 Ros2 토픽으로 변환하여 퍼블리시 하도록, robots/lekiwi/config_lekiwi.py, robots/lekiwi/lekiwi_host.py 를 참고하여 lerobot 플랫폼에서 사용하는 omniwheel IK 를 isaac sim 에 맞게 수정하여 키보드의 입력값을 휠의 출력으로 변환할 수 있게 작성하였다

```
def _kiwi_body_to_wheel_radps(self, vx, vy, wz):
    angles = np.radians([-30.0, -150.0, 90.0])
    M = np.array([[cos(a), sin(a), base_radius] for a in angles])
    wheel_radps = M.dot([vx, vy, wz]) / wheel_radius
```

joint_state_reader_with_wheel.py (전체 코드)

```
#!/usr/bin/env python3
# tcp_joint_state_reader.py (UPDATED: publish wheel command from keyboard/base teleop)
# Home Ubuntu: TCP(JSON lines) -> ROS2 publish:
# - /joint_states (arm positions) [same as before]
# - /wheel_cmds (wheel velocities rad/s) [NEW, matches your old wheel test]

import argparse
import json
import math
import socket
import threading
import time
from typing import List, Optional, Tuple, Dict

import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class TcpJointStateReader(Node):
    def __init__(
        self,
        listen_ip: str,
        listen_port: int,
        arm_topic: str,
        pub_hz: float,
        wheel_topic: str,
        wheel_joints: List[str],
        wheel_radius: float,
        base_radius: float,
        wheel_signs: List[float],
        base_watchdog_s: float,
    ):
        super().__init__("tcp_joint_state_reader")

        # arm publisher (existing)
        self.pub = self.create_publisher(JointState, arm_topic, 10)
        self.topic = arm_topic
        self.pub_hz = pub_hz

        # wheel publisher (new)
        self.wheel_pub = self.create_publisher(JointState, wheel_topic, 10)
        self.wheel_topic = wheel_topic

        # IMPORTANT: wheel order must match your IK mapping
        # order: [front-left, front-right, back]
        self.wheel_joints = wheel_joints
        self.wheel_radius = float(wheel_radius) # meters
        self.base_radius = float(base_radius) # meters
        self.wheel_signs = wheel_signs # [1,1,1] or [-1,...]
        self.base_watchdog_s = float(base_watchdog_s)

        self._lock = threading.Lock()
        self._latest_arm: Optional[Tuple[List[str], List[float], float]] = None # (names, pos, t)
        self._latest_base: Optional[Tuple[float, float, float, float]] = None # (vx, vy, wz, t)

        self.last_rx_time = 0.0
        self.rx_count = 0
        self.rx_count_prev = 0

        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

[illegible]

```

        self._latest_arm = (list(names), pos_f, now)
    except Exception:
        pass

    # update base if valid
    if isinstance(base, dict):
        try:
            vx = float(base.get("vx", 0.0)) # m/s
            vy = float(base.get("vy", 0.0)) # m/s
            wz = float(base.get("wz", 0.0)) # rad/s
            self._latest_base = (vx, vy, wz, now)
        except Exception:
            pass

    self.last_rx_time = now
    self.rx_count += 1

    except socket.timeout:
        continue
    except Exception:
        break
    finally:
        try:
            conn.close()
        except Exception:
            pass
        self.get_logger().warn("    TCP client disconnected (waiting again)")

def _kiwi_body_to_wheel_radps(self, vx: float, vy: float, wz: float) -> List[float]:
    """
    Kiwi 3-omni base IK.
    Returns wheel angular velocities (rad/s) in the order:
    [front-left (axle_2 joint), front-right (axle_1 joint), back (axle_0 joint)]
    Using your R, L and the same structure as your old wheel test.
    """
    v = np.array([vx, vy, wz], dtype=float)

    # NOTE: If your old autopilot mapping was:
    # v1 = (sqrt(3)/2)vx - 0.5 vy + L*wz
    # v2 = -(sqrt(3)/2)vx - 0.5 vy + L*wz
    # v3 = vy + L*wz
    # then angles are [-30°, -150°, 90°] for [v1,v2,v3].
    angles = np.radians(np.array([-30.0, -150.0, 90.0]))
    M = np.array([[math.cos(a), math.sin(a), self.base_radius] for a in angles], dtype=float)

    wheel_linear = M.dot(v) # m/s
    wheel_radps = wheel_linear / self.wheel_radius
    wheel_radps = wheel_radps * np.array(self.wheel_signs, dtype=float)
    return [float(x) for x in wheel_radps]

def _publish_latest(self):
    now = time.time()

    with self._lock:
        arm = self._latest_arm
        base = self._latest_base

    # ---- publish arm (same as before) ----
    if arm is not None:
        names, positions, _t = arm
        js = JointState()
        js.header.stamp = self.get_clock().now().to_msg()
        js.name = names
        js.position = positions

```

```

        self.pub.publish(js)

# ---- publish wheels (new) ----
vx = vy = wz = 0.0
if base is not None:
    _vx, _vy, _wz, t_base = base
    if (now - t_base) <= self.base_watchdog_s:
        vx, vy, wz = _vx, _vy, _wz
        wz=-wz

wheel_vel = self._kiwi_body_to_wheel_radps(vx, vy, wz)

wmsg = JointState()
wmsg.header.stamp = self.get_clock().now().to_msg()
wmsg.name = list(self.wheel_joints)
wmsg.velocity = wheel_vel
self.wheel_pub.publish(wmsg)

def _print_rx_rate(self):
    with self._lock:
        c = self.rx_count
        last_t = self.last_rx_time

    hz = c - self._rx_count_prev
    self._rx_count_prev = c

    if last_t > 0 and (time.time() - last_t) < 2.0:
        self.get_logger().info(f"    rx ~{hz} pkt/s (stream alive)")
    else:
        self.get_logger().warn("... no rx (stream dead or not connected)")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--listen-ip", default="0.0.0.0")
    ap.add_argument("--listen-port", type=int, default=15002)

    # existing arm publish
    ap.add_argument("--topic", default="/leader_joint_states")
    ap.add_argument("--hz", type=float, default=30.0)

    # new wheel publish (default matches your old test)
    ap.add_argument("--wheel-topic", default="/wheel_cmds")

    #   HERE: your exact joint names (front-left, front-right, back)
    ap.add_argument(
        "--wheel-joints",
        nargs=3,
        default=[
            "axle_2_joint", # 전방 왼쪽 (Left)
            "axle_1_joint", # 전방 오른쪽 (Right)
            "axle_0_joint", # 후방 (Back)
        ],
        help="3 wheel joint names in order: front-left front-right back (default: axle_2_joint axle_1_joint axle_0_joint)",
    )

    ap.add_argument("--wheel-radius", type=float, default=0.0437)
    ap.add_argument("--base-radius", type=float, default=0.1214315)

    ap.add_argument(
        "--wheel-signs",
        nargs=3,
        type=float,

```

```

        default=[1.0, 1.0, 1.0],
        help="sign flips for each wheel: front-left front-right back (use -1 if a wheel spins opposite)",
    )

    ap.add_argument(
        "--base-watchdog",
        type=float,
        default=0.5,
        help="if base cmd not received within this time -> publish zero wheel velocities",
    )

    args = ap.parse_args()

    rclpy.init()
    node = TcpJointStateReader(
        listen_ip=args.listen_ip,
        listen_port=args.listen_port,
        arm_topic=args.topic,
        pub_hz=args.hz,
        wheel_topic=args.wheel_topic,
        wheel_joints=list(args.wheel_joints),
        wheel_radius=float(args.wheel_radius),
        base_radius=float(args.base_radius),
        wheel_signs=list(args.wheel_signs),
        base_watchdog_s=float(args.base_watchdog),
    )
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        try:
            node.server.close()
        except Exception:
            pass
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

source /opt/ros/humble/setup.bash
sudo ufw allow in on tailscale0 to any port 15002 proto tcp
cd ros2_ws
python3 joint_state_reader_with_wheel.py --listen-port 15002

```

4.3 송신기

송신기의 코드는 기존 lerobot 플랫폼에서 제공하는 `examples/lekiwi/teleoperate.py` 에서 통신 방식을 TCP Socket 으로 변환하고 Isaac sim 에서 사용하는 관절 단위는 라디안이기 때문에 입력값 라디안 변환, 기존 캘리브레이션 진행 시 측정하지 않은 wrist roll 에 대한 배울 적용을 수정한 코드이다

Leader_to_home_with_wheel.py (전체 코드)

```

#!/usr/bin/env python3
# leader_to_home_tcp_rest_matched_with_keyboard_base.py
# Windows: SO100 Leader -> TCP(JSON lines) to Home
# - arm: name/position (same as before)
# - base: {"vx","vy","wz"} (key-state based, sent EVERY tick)

import json
import math
import socket
import time

```

```

from typing import Any, Dict, List, Optional, Tuple

from lerobot.teleoperators.so100_leader import SO100Leader, SO100LeaderConfig
from lerobot.utils.robot_utils import precise_sleep

# ----- keyboard (pynput) -----
try:
    from pynput import keyboard
    PYNPUT_OK = True
except Exception:
    keyboard = None
    PYNPUT_OK = False

FPS = 20

HOME_TAILSCALE_IP = "100.91.14.65"
HOME_TCP_PORT = 15002

JOINT_NAMES = [
    "STS3215_03a_v1_Revolute_45",
    "STS3215_03a_v1_1_Revolute_49",
    "STS3215_03a_v1_2_Revolute_51",
    "STS3215_03a_v1_3_Revolute_53",
    "STS3215_03a_Wrist_Roll_v1_Revolute_55",
    "STS3215_03a_v1_4_Revolute_57",
]

# Measured: Isaac Sim joint positions at Play with no input
SIM_REST_RAD6 = [
    -0.001634, # shoulder_pan
    -0.002328, # shoulder_lift
    0.098572, # elbow_flex
    0.004954, # wrist_flex
    0.009319, # wrist_roll
    -0.000285, # gripper
]

# Measured: Leader arm positions when matching Isaac Sim rest pose
LEADER_REST_RAD6 = [
    0.042740, # shoulder_pan (2.45 deg)
    -1.521986, # shoulder_lift (-87.20 deg)
    1.739000, # elbow_flex (99.64 deg)
    1.201714, # wrist_flex (68.85 deg)
    -0.926154, # wrist_roll (-53.06 deg)
    0.215753, # gripper (18.09 deg)
]

SIGNS = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

SMOOTH_ALPHA = 0.12
MAX_SPEED_RAD_S = 2.0
RAMP_SECONDS = 0.8

# ----- base keymap -----
LINEAR_SPEED = 0.25 # m/s
ANGULAR_SPEED = 2.0 # rad/s
SPEED_STEP = 0.02 # m/s step
ANG_STEP = 0.5 # rad/s step
MIN_LINEAR = 0.02
MIN_ANG = 0.5
MAX_LINEAR = 0.5
MAX_ANG = 10.0

pressed: Dict[str, bool] = {}

```



```

prev_pressed: Dict[str, bool] = {}

def _norm_key_char(k) -> Optional[str]:
    try:
        if hasattr(k, "char") and k.char is not None:
            return str(k.char).lower()
    except Exception:
        pass
    return None

def on_press(k):
    kc = _norm_key_char(k)
    if kc:
        pressed[kc] = True

def on_release(k):
    kc = _norm_key_char(k)
    if kc:
        pressed[kc] = False

def _connect_tcp() -> socket.socket:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5.0)
    s.connect((HOME_TAILSCALE_IP, HOME_TCP_PORT))
    s.settimeout(None)
    s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    return s

def wrap_pi(x: float) -> float:
    return (x + math.pi) % (2.0 * math.pi) - math.pi

def extract_leader_deg(raw_action: Any) -> List[float]:
    deg6 = [0.0] * 6
    if isinstance(raw_action, dict):
        def pick(name: str) -> float:
            for k, v in raw_action.items():
                k2 = str(k).strip().lower().replace(" ", "_").replace("-", "_")
                if k2.startswith("arm_"):
                    k2 = k2[4:]
                if k2.endswith(".pos"):
                    k2 = k2[:-4]
                if k2 == name:
                    try:
                        return float(v)
                    except Exception:
                        return 0.0
            return 0.0

        deg6[0] = pick("shoulder_pan")
        deg6[1] = pick("shoulder_lift")
        deg6[2] = pick("elbow_flex")
        deg6[3] = pick("wrist_flex")
        deg6[4] = pick("wrist_roll")
        deg6[5] = pick("gripper")
    return deg6

def leader_deg_to_rad6(deg6: List[float]) -> List[float]:
    rad6 = [math.radians(x) for x in deg6]
    rad6[0] = wrap_pi(rad6[0])
    return rad6

def rate_limit(cur: float, prev: float, max_speed: float, dt: float) -> float:
    if dt <= 0:
        return cur

```

```

max_step = max_speed * dt
d = cur - prev
if d > max_step:
    return prev + max_step
if d < -max_step:
    return prev - max_step
return cur

def compute_base_cmd() -> Dict[str, float]:
    global LINEAR_SPEED, ANGULAR_SPEED, prev_pressed

    active = {k for k, v in pressed.items() if v}

    def rising(k: str) -> bool:
        return (k in active) and (not prev_pressed.get(k, False))

    if rising("r"):
        LINEAR_SPEED = min(MAX_LINEAR, LINEAR_SPEED + SPEED_STEP)
        ANGULAR_SPEED = min(MAX_ANG, ANGULAR_SPEED + ANG_STEP)
        print(f"[BASE] speed up -> linear={LINEAR_SPEED:.3f} m/s, ang={ANGULAR_SPEED:.3f} rad/s")
    if rising("f"):
        LINEAR_SPEED = max(MIN_LINEAR, LINEAR_SPEED - SPEED_STEP)
        ANGULAR_SPEED = max(MIN_ANG, ANGULAR_SPEED - ANG_STEP)
        print(f"[BASE] speed down -> linear={LINEAR_SPEED:.3f} m/s, ang={ANGULAR_SPEED:.3f} rad/s")

    prev_pressed = {k: (k in active) for k in ["w", "a", "s", "d", "z", "x", "r", "f"]}

    vx = vy = wz = 0.0
    if "w" in active: vx += LINEAR_SPEED
    if "s" in active: vx -= LINEAR_SPEED
    if "a" in active: vy += LINEAR_SPEED
    if "d" in active: vy -= LINEAR_SPEED
    if "z" in active: wz += ANGULAR_SPEED
    if "x" in active: wz -= ANGULAR_SPEED

    return {"vx": float(vx), "vy": float(vy), "wz": float(wz)}

def main():
    leader = SO100Leader(SO100LeaderConfig(port="COM8", id="my_awesome_leader_arm"))
    leader.connect()
    if not leader.is_connected:
        raise RuntimeError("Leader arm not connected")

    if PYNPUT_OK:
        listener = keyboard.Listener(on_press=on_press, on_release=on_release)
        listener.start()
        print("[KEY] pynput listener started")
    else:
        print("[KEY] pynput NOT available. base cmd will stay zero.")

    print(f"Target TCP = {HOME_TAILSCALE_IP}:{HOME_TCP_PORT}")
    print("SIM_REST_RAD6 =", [round(x, 6) for x in SIM_REST_RAD6])
    print("LEADER_REST_RAD6 =", [round(x, 6) for x in LEADER_REST_RAD6])
    print("SIGNS =", SIGNS)

    sock: Optional[socket.socket] = None
    last_out: Optional[List[float]] = None
    last_t = time.time()
    t_stream_start: Optional[float] = None

    while True:
        if sock is None:
            try:
                sock = _connect_tcp()

```

```

        t_stream_start = time.time()
        print("CONNECTED")
    except Exception as e:
        print("connect failed:", e)
        time.sleep(1.0)
        continue

    tick_start = time.perf_counter()

    # ----- arm -----
    deg6 = extract_leader_deg(leader.get_action())
    leader_rad6 = leader_deg_to_rad6(deg6)

    delta = [leader_rad6[i] - LEADER_REST_RAD6[i] for i in range(6)]
    target = [SIM_REST_RAD6[i] + SIGNS[i] * delta[i] for i in range(6)]

    if t_stream_start is not None and RAMP_SECONDS > 0:
        a = min(max((time.time() - t_stream_start) / RAMP_SECONDS, 0.0), 1.0)
        target = [SIM_REST_RAD6[i] + a * (target[i] - SIM_REST_RAD6[i]) for i in range(6)]

    now = time.time()
    dt = now - last_t
    last_t = now

    if last_out is None:
        out = list(target)
    else:
        tmp = [rate_limit(target[i], last_out[i], MAX_SPEED_RAD_S, dt) for i in range(6)]
        a = SMOOTH_ALPHA
        out = [(1 - a) * last_out[i] + a * tmp[i] for i in range(6)]
    last_out = list(out)

    # ----- base -----
    base_cmd = compute_base_cmd() if PYNPUT_OK else {"vx": 0.0, "vy": 0.0, "wz": 0.0}

    payload = {
        "t": time.time(),
        "name": JOINT_NAMES,
        "position": out,
        "base": base_cmd,
        "src": "so100_leader+keyboard",
    }

    try:
        sock.sendall((json.dumps(payload) + "\n").encode("utf-8"))
    except Exception as e:
        print("send failed:", e)
        try:
            sock.close()
        except Exception:
            pass
        sock = None
        continue

    precise_sleep(max(1.0 / FPS - (time.perf_counter() - tick_start), 0.0))

if __name__ == "__main__":
    main()

python .\yourpath\Leader_to_home_with_wheel.py

```

5. 물체 스폰

5.0 isaac sim object 추가

데이터를 수집하거나 로봇을 검증할 때 시뮬레이터를 사용하는 가장 큰 이유중 하나는 시뮬레이터에서는 현실과 달리 다양한 물체를 쉽게 활용할 수 있다는 것이다.

이를 위해 google 이 제공하는 1030 종의 실제 물건 정밀 스캔 데이터인 google scanned objects 를 또다른 시뮬레이터 Mujoco 에서 스폰하여 사용할 수 있는 깃허브 레퍼지토리를 활용한다.

```
git clone https://github.com/kevinzakka/mujoco_scanned_objects.git
cd mujoco_scanned_objects
```

하지만 Mujoco 에서는 xml 형식을 사용하는 반면 isaac sim 에서는 usd 형식을 사용하기 때문에 형식 변환을 진행해야 한다

xml→usd 변환 스크립트 (전체 코드)

```
import os
import json
import traceback
import asyncio
from pathlib import Path

import omni
import omni.kit.asset_converter
from pxr import Usd, UsdGeom, UsdPhysics, UsdShade, Sdf, Gf

# =====
# 설정
# =====
REPO_ROOT = Path("/home/yubin/isaac-objects/mujoco_scanned_objects")
MODELS_DIR = REPO_ROOT / "models"

# 전체용 인덱스(새로 생성)
INDEX_PATH_ALL = Path("/home/yubin/isaac-objects/mujoco_obj_usd_index_all.jsonl")

# 처리 개수 (0 이면 전부 = 1030 개)
N_OBJECTS = 0

# 출력 USD 이름(스폰 스크립트가 이 이름을 참조하니 그대로 유지)
RENDER_USD_NAME = "model_clean.usd"

# 콜라이더 USD 저장 폴더
COLLIDER_DIR_NAME = "colliders"
COLLIDER_USD_FMT = "collider_{:02d}.usd"

# 물리
MASS = 0.5

# 변환 옵션
CONVERTER_SINGLE_MESH = True
EMBED_TEXTURES = True

# =====
# 변환 유틸
# =====
async def convert_obj_to_usd(obj_path: str, usd_path: str) -> bool:
    converter_manager = omni.kit.asset_converter.get_instance()

    context = omni.kit.asset_converter.AssetConverterContext()
    context.ignore_materials = False
    context.ignore_animations = True
    context.ignore_cameras = True
```

```

context.single_mesh = bool(CONVERTER_SINGLE_MESH)
context.smooth_normals = True
context.export_preview_surface = True
context.use_meter_as_world_unit = True
context.embed_textures = bool(EMBED_TEXTURES)

task = converter_manager.create_converter_task(obj_path, usd_path, None, context)
success = await task.wait_until_finished()
return bool(success)

def apply_texture_to_usd(usd_path: str, texture_path: str):
    if not os.path.exists(texture_path):
        return

    stage = Usd.Stage.Open(usd_path)

    root_prim = stage.GetDefaultPrim()
    if not root_prim:
        for prim in stage.Traverse():
            if prim.IsA(UsdGeom.Xform):
                root_prim = prim
                stage.SetDefaultPrim(root_prim)
                break
    if not root_prim:
        stage.Save()
        return

    root_path = root_prim.GetPath().pathString
    looks_path = f"{root_path}/Looks"
    mat_path = f"{looks_path}/Material"

    if not stage.GetPrimAtPath(looks_path):
        stage.DefinePrim(looks_path, "Scope")

    if stage.GetPrimAtPath(mat_path):
        stage.RemovePrim(mat_path)

    material = UsdShade.Material.Define(stage, mat_path)

    shader_path = f"{mat_path}/PBRShader"
    shader = UsdShade.Shader.Define(stage, shader_path)
    shader.CreateIdAttr("UsdPreviewSurface")
    shader.CreateInput("roughness", Sdf.ValueTypeNames.Float).Set(0.5)
    shader.CreateInput("metallic", Sdf.ValueTypeNames.Float).Set(0.0)

    tex_sampler_path = f"{mat_path}/DiffuseTextureSampler"
    tex_sampler = UsdShade.Shader.Define(stage, tex_sampler_path)
    tex_sampler.CreateIdAttr("UsdUVTexture")
    tex_sampler.CreateInput("file", Sdf.ValueTypeNames.Asset).Set(str(texture_path))
    tex_sampler.CreateInput("wrapS", Sdf.ValueTypeNames.Token).Set("repeat")
    tex_sampler.CreateInput("wrapT", Sdf.ValueTypeNames.Token).Set("repeat")
    tex_sampler.CreateInput("sourceColorSpace", Sdf.ValueTypeNames.Token).Set("sRGB")

    tex_output = tex_sampler.CreateOutput("rgb", Sdf.ValueTypeNames.Float3)

    uv_reader_path = f"{mat_path}/UVReader"
    uv_reader = UsdShade.Shader.Define(stage, uv_reader_path)
    uv_reader.CreateIdAttr("UsdPrimvarReader_float2")
    uv_reader.CreateInput("varname", Sdf.ValueTypeNames.Token).Set("st")
    uv_output = uv_reader.CreateOutput("result", Sdf.ValueTypeNames.Float2)

    tex_sampler.CreateInput("st", Sdf.ValueTypeNames.Float2).ConnectToSource(uv_output)
    shader.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).ConnectToSource(tex_output)

```

```

material.CreateSurfaceOutput().ConnectToSource(shader.ConnectableAPI(), "surface")

for prim in stage.Traverse():
    if prim.IsA(UsdGeom.Mesh):
        UsdShade.MaterialBindingAPI.Apply(prim).Bind(material)
        # 내부가 보이게(시각용). 충돌과는 무관.
        UsdGeom.Mesh(prim).CreateDoubleSidedAttr(True)

stage.Save()

def setup_rigidbody_on_root(stage: Usd.Stage, mass: float):
    root_prim = stage.GetDefaultPrim()
    if not root_prim:
        for prim in stage.Traverse():
            if prim.IsA(UsdGeom.Xform):
                root_prim = prim
                stage.SetDefaultPrim(root_prim)
                break
    if not root_prim:
        return None

    if not root_prim.HasAPI(UsdPhysics.RigidBodyAPI):
        UsdPhysics.RigidBodyAPI.Apply(root_prim)

    if not root_prim.HasAPI(UsdPhysics.MassAPI):
        mass_api = UsdPhysics.MassAPI.Apply(root_prim)
        mass_api.CreateMassAttr(float(mass))

    return root_prim

def attach_collision_usds_into_render(render_usd_path: Path, collider_usd_paths: list) -> bool:
    stage = Usd.Stage.Open(str(render_usd_path))

    root_prim = setup_rigidbody_on_root(stage, MASS)
    if root_prim is None:
        stage.Save()
        return False

    root_path = root_prim.GetPath().pathString
    coll_root_path = f"{root_path}/Colliders"
    if not stage.GetPrimAtPath(coll_root_path):
        stage.DefinePrim(coll_root_path, "Xform")

    # 콜라이더는 안 보이게
    UsdGeom.Imageable(stage.GetPrimAtPath(coll_root_path)).CreateVisibilityAttr("invisible")

    # collider USD 들을 참조로 붙임
    for i, col_usd in enumerate(collider_usd_paths):
        child_path = f"{coll_root_path}/C{i:02d}"
        child = stage.DefinePrim(child_path, "Xform")
        child.GetReferences().AddReference(str(col_usd))

    # collider 아래 Mesh 에만 collision 적용
    for prim in stage.Traverse():
        p = prim.GetPath().pathString
        if not p.startswith(coll_root_path):
            continue
        if prim.IsA(UsdGeom.Mesh):
            UsdPhysics.CollisionAPI.Apply(prim)
            mc = UsdPhysics.MeshCollisionAPI.Apply(prim)
            mc.CreateApproximationAttr().Set("convexHull")
            UsdGeom.Imageable(prim).CreateVisibilityAttr("invisible")

    stage.Save()

```

```

return True

def fallback_apply_convex_decomposition_on_render(render_usd_path: Path) -> bool:
    # collision submesh 가 없을 때의 폴백: 렌더 메쉬에 convexDecomposition
    stage = Usd.Stage.Open(str(render_usd_path))
    root_prim = setup_rigidbody_on_root(stage, MASS)
    if root_prim is None:
        stage.Save()
        return False

    for prim in stage.Traverse():
        if prim.IsA(UsdGeom.Mesh):
            UsdPhysics.CollisionAPI.Apply(prim)
            mc = UsdPhysics.MeshCollisionAPI.Apply(prim)
            mc.CreateApproximationAttr().Set("convexDecomposition")

    stage.Save()
    return True

# =====
# 메인
# =====
async def main():
    obj_files = sorted(MODELS_DIR.glob("*/model.obj"))
    if not obj_files:
        raise RuntimeError("OBJ 파일을 찾을 수 없음")

    selected = obj_files if N_OBJECTS == 0 else obj_files[:min(N_OBJECTS, len(obj_files))]
    print(f"=== 전체 OBJ → USD (렌더+collision) 시작: {len(selected)}개 ===")

    ok = 0
    with INDEX_PATH_ALL.open("w", encoding="utf-8") as fidx:
        for i, obj_path in enumerate(selected, 1):
            obj_dir = obj_path.parent
            obj_name = obj_dir.name

            render_usd = obj_dir / RENDER_USD_NAME
            texture_path = obj_dir / "texture.png"

            collision_objs = sorted(obj_dir.glob("model_collision_*.obj")) # 보통 32 개
            collider_dir = obj_dir / COLLIDER_DIR_NAME
            collider_dir.mkdir(parents=True, exist_ok=True)

            try:
                print(f"[{i}/{len(selected)}] {obj_name} ...", end=" ")

                # 1) 렌더 USD 생성
                if render_usd.exists():
                    try:
                        os.remove(render_usd)
                    except:
                        pass

                s1 = await convert_obj_to_usd(str(obj_path), str(render_usd))
                if not s1 or not render_usd.exists():
                    print("렌더 변환 실패")
                    continue

                if texture_path.exists():
                    apply_texture_to_usd(str(render_usd), str(texture_path))

                # 2) collision obj → collider usd 생성
                collider_usds = []
                for k, col_obj in enumerate(collision_objs):

```

```

        out_col_usd = collider_dir / COLLIDER_USD_FMT.format(k)
        if out_col_usd.exists():
            try:
                os.remove(out_col_usd)
            except:
                pass
        s2 = await convert_obj_to_usd(str(col_obj), str(out_col_usd))
        if s2 and out_col_usd.exists():
            collider_usds.append(out_col_usd)

# 3) 렌더 USD 에 collider 붙이기 (없으면 폴백)
if collider_usds:
    attached = attach_collision_usds_into_render(render_usd, collider_usds)
    if not attached:
        print("collider attach 실패")
        continue
    note = "render=model.obj, collider=model_collision_*.obj"
else:
    fb = fallback_apply_convex_decomposition_on_render(render_usd)
    if not fb:
        print("fallback 실패")
        continue
    note = "render=model.obj, collider=fallback(convexDecomposition)"

print("ok")
fidx.write(json.dumps({
    "name": obj_name,
    "usd": str(render_usd),
    "status": "ok",
    "note": note
}, ensure_ascii=False) + "\n")
ok += 1

except Exception:
    print("실패")
    traceback.print_exc()

print(f"\n[완료] 성공: {ok}개")
print(f"[인덱스] {INDEX_PATH_ALL}")

asyncio.ensure_future(main())

```

5.1 isaac sim object 스폰

windows->script editor 에서 아래의 물체 스폰 코드를 입력하여 새로 추가한 1030 개의 물품을 랜덤으로 스폰하고 이를 담은 바구니를 스폰할 수 있다

물체 스폰 script editor (전체 코드)

```

"""
물체/타겟 스폰 스크립트 (Isaac Sim Script Editor 용)
- Domain Randomization (조명)
- 물체 스케일: 0.7 고정
- 타겟(초록 바구니): Dynamic RigidBody + Colliders (움직이게)
- PhysicsScene + Timeline Play 자동 보장
- Ground 생성/콜라이더 추가 없음 (사용자 씬의 기존 바닥을 그대로 사용)
"""

import json
import random
import math
from pathlib import Path

import omni.usd

```



```

import omni.timeline
from pxr import UsdGeom, Gf, UsdLux, UsdPhysics, PhysxSchema, Sdf

# =====
# 설정
# =====
OBJECTS_INDEX_PATH = Path("/home/yubin11/isaac-objects/mujoco_obj_usd_index_all.jsonl")

# 스폰 영역 (로봇 앞쪽)
SPAWN_RANGE_X = (-2.0, 2.0)
SPAWN_RANGE_Y = (-2.0, 2.0)
SPAWN_HEIGHT = 0.05

# 타겟(바구니) 영역
TARGET_RANGE_X = (-2.0, 2.0)
TARGET_RANGE_Y = (-2.0, 2.0)
TARGET_HEIGHT = 0.0

# 경로
SPAWNED_OBJECT_PATH = "/World/SpawnedObject"
TARGET_MARKER_PATH = "/World/TargetMarker"
DR_LIGHTS_PATH = "/World/DR_Lights"

# 물체 스케일 고정
OBJECT_FIXED_SCALE = 0.7

# 바구니 물리 파라미터
BASKET_DYNAMIC = True
BASKET_MASS = 1.0
BASKET_ENABLE_CCD = True
BASKET_LINEAR_DAMPING = 0.05
BASKET_ANGULAR_DAMPING = 0.1

# PhysicsScene 자동 생성
PHYSICS_SCENE_PATH = "/World/physicsScene"
GRAVITY = 9.81 # m/s^2

# Domain Randomization 설정
DR_CONFIG = {
    "lighting": {
        "intensity_range": (800.0, 1500.0),
        "color_temp_range": (5400, 7500),
        "position_z_range": (2.0, 3.0),
        "num_lights_range": (1, 2),
    },
    "object": {
        "rotation_z_range": (0, 360),
    },
},

# 연두색 바구니 설정
"target_basket": {
    "basket_color_name": "lime_green_basket",
    "basket_color_rgb": (0.5, 0.9, 0.3),

    # 바구니 크기
    "basket_width": 0.25, # 가로 25cm
    "basket_depth": 0.18, # 세로 18cm
    "basket_height": 0.12, # 높이 12cm
    "wall_thickness": 0.008, # 벽 두께 8mm
    "bottom_thickness": 0.01, # 바닥 두께 1cm

    # 손잡이
    "handle_enable": True,
    "handle_radius": 0.008,

```

```

        "handle_height": 0.06,
    },
}

# =====
# 유틸리티
# =====
def safe_prim_name(name: str) -> str:
    """USD 호환 이름으로 변환"""
    safe = "".join(c if c.isalnum() else '_' for c in name)[:40]
    if safe and safe[0].isdigit():
        safe = '_' + safe
    return safe or "object"

def kelvin_to_rgb(kelvin: float):
    """색온도를 RGB 로 변환"""
    temp = kelvin / 100.0

    if temp <= 66:
        red = 255
        green = 99.4708 * math.log(temp) - 161.1195
    else:
        red = 329.6987 * ((temp - 60) ** -0.1332)
        green = 288.1222 * ((temp - 60) ** -0.0755)

    if temp >= 66:
        blue = 255
    elif temp <= 19:
        blue = 0
    else:
        blue = 138.5177 * math.log(temp - 10) - 305.0448

    return (
        max(0, min(255, red)) / 255.0,
        max(0, min(255, green)) / 255.0,
        max(0, min(255, blue)) / 255.0
    )

def clear_spawned():
    """기존 스폰 물체 삭제"""
    stage = omni.usd.get_context().get_stage()
    for path in [SPAWNED_OBJECT_PATH, TARGET_MARKER_PATH, DR_LIGHTS_PATH]:
        prim = stage.GetPrimAtPath(path)
        if prim and prim.IsValid():
            stage.RemovePrim(path)

def ensure_physics_scene():
    """PhysicsScene(중력) 없으면 생성"""
    stage = omni.usd.get_context().get_stage()

    scene_prim = stage.GetPrimAtPath(PHYSICS_SCENE_PATH)
    if not scene_prim or not scene_prim.IsValid():
        scene = UsdPhysics.Scene.Define(stage, PHYSICS_SCENE_PATH)
        scene_prim = scene.GetPrim()
        print(f"[Physics] Created: {PHYSICS_SCENE_PATH}")
    else:
        scene = UsdPhysics.Scene(scene_prim)
        print(f"[Physics] Found: {PHYSICS_SCENE_PATH}")

    scene.CreateGravityDirectionAttr().Set(Gf.Vec3f(0.0, 0.0, -1.0))
    scene.CreateGravityMagnitudeAttr().Set(float(GRAVITY))

```

```

# PhysX Scene API (옵션)
PhysxSchema.PhysxSceneAPI.Apply(scene_prim)
physx_api = PhysxSchema.PhysxSceneAPI.Get(stage, Sdf.Path(PHYSICS_SCENE_PATH))
physx_api.CreateEnableGPUDynamicsAttr(True)

def ensure_timeline_play():
    """시뮬레이션 Play 보장"""
    tl = omni.timeline.get_timeline_interface()
    if not tl.is_playing():
        tl.play()
        print("[Timeline] PLAY")

def randomize_lighting():
    """조명 랜덤화"""
    stage = omni.usd.get_context().get_stage()
    config = DR_CONFIG["lighting"]

    if not stage.GetPrimAtPath(DR_LIGHTS_PATH):
        stage.DefinePrim(DR_LIGHTS_PATH, "Xform")

    num_lights = random.randint(*config["num_lights_range"])

    for i in range(num_lights):
        light_path = f"{DR_LIGHTS_PATH}/Light_{i}"
        light = UsdLux.RectLight.Define(stage, light_path)

        intensity = random.uniform(*config["intensity_range"])
        light.GetIntensityAttr().Set(float(intensity))

        color_temp = random.uniform(*config["color_temp_range"])
        rgb = kelvin_to_rgb(color_temp)
        light.GetColorAttr().Set(Gf.Vec3f(*rgb))

        light.GetWidthAttr().Set(float(random.uniform(0.8, 1.2)))
        light.GetHeightAttr().Set(float(random.uniform(0.2, 0.4)))

        x = random.uniform(-1.0, 1.0)
        y = random.uniform(-1.0, 1.0)
        z = random.uniform(*config["position_z_range"])

        xform = UsdGeom.Xformable(light)
        xform.ClearXformOpOrder()
        xform.AddTranslateOp().Set(Gf.Vec3d(float(x), float(y), float(z)))
        xform.AddRotateXYZOp().Set(Gf.Vec3f(-90.0, 0.0, float(random.uniform(0, 360))))

    dome_path = f"{DR_LIGHTS_PATH}/Dome"
    dome = UsdLux.DomeLight.Define(stage, dome_path)
    dome.GetIntensityAttr().Set(float(random.uniform(200, 400)))
    dome.GetColorAttr().Set(Gf.Vec3f(1.0, 1.0, 1.0))

    print(f"[DR] 조명 {num_lights}개 생성")

def apply_collider(prim):
    """프림에 collider 부여"""
    if prim and prim.IsValid() and not prim.HasAPI(UsdPhysics.CollisionAPI):
        UsdPhysics.CollisionAPI.Apply(prim)

# =====
# 스폰 함수들

```

```

# =====
def spawn_object(objects_list: list):
    """랜덤 물체 스폰 ( 스케일 0.7 고정)"""
    if not objects_list:
        print("[ERROR] 물체 리스트가 비어있음")
        return None, None, None

    stage = omni.usd.get_context().get_stage()
    config = DR_CONFIG["object"]

    obj_info = random.choice(objects_list)
    obj_name = obj_info["name"]
    obj_usd = obj_info["usd"]

    x = random.uniform(*SPAWN_RANGE_X)
    y = random.uniform(*SPAWN_RANGE_Y)
    z = float(SPAWN_HEIGHT)

    rot_z = random.uniform(*config["rotation_z_range"])

    safe_name = safe_prim_name(obj_name)
    prim_path = f"{SPAWNED_OBJECT_PATH}/{safe_name}"

    if not stage.GetPrimAtPath(SPAWNED_OBJECT_PATH):
        stage.DefinePrim(SPAWNED_OBJECT_PATH, "Xform")

    prim = stage.DefinePrim(prim_path, "Xform")
    prim.GetReferences().AddReference(obj_usd)

    xformable = UsdGeom.Xformable(prim)
    xformable.ClearXformOpOrder()
    xformable.AddTranslateOp().Set(Gf.Vec3d(float(x), float(y), float(z)))
    xformable.AddRotateXYZOp().Set(Gf.Vec3f(0.0, 0.0, float(rot_z)))

    # 스케일 고정 0.7
    s = float(OBJECT_FIXED_SCALE)
    xformable.AddScaleOp().Set(Gf.Vec3f(s, s, s))

    return obj_name, (x, y, z), s

def spawn_target(avoid_pos=None):
    """타겟: 연두색 바구니 스폰 ( Dynamic RigidBody + Colliders)"""
    stage = omni.usd.get_context().get_stage()
    cfg = DR_CONFIG["target_basket"]

    # 위치 (물체와 최소 거리 유지)
    x, y = None, None
    for _ in range(20):
        xx = random.uniform(*TARGET_RANGE_X)
        yy = random.uniform(*TARGET_RANGE_Y)
        if avoid_pos:
            dist = ((xx - avoid_pos[0])**2 + (yy - avoid_pos[1])**2) ** 0.5
            if dist >= 0.15:
                x, y = xx, yy
                break
        else:
            x, y = xx, yy
            break

    if x is None:
        x = random.uniform(*TARGET_RANGE_X)
        y = random.uniform(*TARGET_RANGE_Y)

```

```

z = float(TARGET_HEIGHT)

# 바구니 파라미터
bw = float(cfg["basket_width"])
bd = float(cfg["basket_depth"])
bh = float(cfg["basket_height"])
wt = float(cfg["wall_thickness"])
bt = float(cfg["bottom_thickness"])

basket_rgb = cfg["basket_color_rgb"]

# 그룹(Xform)
marker = UsdGeom.Xform.Define(stage, TARGET_MARKER_PATH)
marker_xf = UsdGeom.Xformable(marker)
marker_xf.ClearXformOpOrder()
marker_xf.AddTranslateOp().Set(Gf.Vec3d(float(x), float(y), float(z)))

# ===== 바닥판 =====
bottom = UsdGeom.Cube.Define(stage, f"{TARGET_MARKER_PATH}/Bottom")
bottom.GetSizeAttr().Set(1.0)
bottom.GetDisplayColorAttr().Set([Gf.Vec3f(*basket_rgb)])
bottom.GetDisplayOpacityAttr().Set([1.0])

bottom_xf = UsdGeom.Xformable(bottom)
bottom_xf.ClearXformOpOrder()
bottom_xf.AddTranslateOp().Set(Gf.Vec3d(0.0, 0.0, float(bt * 0.5)))
bottom_xf.AddScaleOp().Set(Gf.Vec3f(float(bw), float(bd), float(bt)))

apply_collider(bottom.GetPrim())

# ===== 4 개의 벽 =====
wall_configs = [
    ("WallFront", (0.0, (bd - wt) * 0.5, bt + bh * 0.5), (bw, wt, bh)),
    ("WallBack", (0.0, -(bd - wt) * 0.5, bt + bh * 0.5), (bw, wt, bh)),
    ("WallLeft", (-(bw - wt) * 0.5, 0.0, bt + bh * 0.5), (wt, bd - 2 * wt, bh)),
    ("WallRight", ((bw - wt) * 0.5, 0.0, bt + bh * 0.5), (wt, bd - 2 * wt, bh)),
]

for wall_name, pos, size in wall_configs:
    wall = UsdGeom.Cube.Define(stage, f"{TARGET_MARKER_PATH}/{wall_name}")
    wall.GetSizeAttr().Set(1.0)
    wall.GetDisplayColorAttr().Set([Gf.Vec3f(*basket_rgb)])
    wall.GetDisplayOpacityAttr().Set([1.0])

    wall_xf = UsdGeom.Xformable(wall)
    wall_xf.ClearXformOpOrder()
    wall_xf.AddTranslateOp().Set(Gf.Vec3d(float(pos[0]), float(pos[1]), float(pos[2])))
    wall_xf.AddScaleOp().Set(Gf.Vec3f(float(size[0]), float(size[1]), float(size[2])))

    apply_collider(wall.GetPrim())

# ===== 손잡이 (선택) =====
if cfg.get("handle_enable", True):
    handle_r = float(cfg.get("handle_radius", 0.008))
    handle_h = float(cfg.get("handle_height", 0.06))

    for side, sx in [("Left", -(bw * 0.5 - wt)), ("Right", (bw * 0.5 - wt))]:
        for hy, name_suffix in [(bd * 0.25, "Front"), (-bd * 0.25, "Back")]:
            post = UsdGeom.Cylinder.Define(stage,
                f"{TARGET_MARKER_PATH}/Handle{side}Post{name_suffix}")
            post.GetAxisAttr().Set("Z")
            post.GetRadiusAttr().Set(float(handle_r))
            post.GetHeightAttr().Set(float(handle_h))
            post.GetDisplayColorAttr().Set([Gf.Vec3f(*basket_rgb)])

```

```

        post.GetDisplayOpacityAttr().Set([1.0])

        post_xf = UsdGeom.Xformable(post)
        post_xf.ClearXformOpOrder()
        post_xf.AddTranslateOp().Set(Gf.Vec3d(float(sx), float(hy), float(bt + bh + handle_h * 0.5)))

        apply_collider(post.GetPrim())

        bar = UsdGeom.Cylinder.Define(stage, f"{TARGET_MARKER_PATH}/Handle{side}Bar")
        bar.GetAxisAttr().Set("Y")
        bar.GetRadiusAttr().Set(float(handle_r))
        bar.GetHeightAttr().Set(float(bd * 0.5))
        bar.GetDisplayColorAttr().Set([Gf.Vec3f(*basket_rgb)])
        bar.GetDisplayOpacityAttr().Set([1.0])

        bar_xf = UsdGeom.Xformable(bar)
        bar_xf.ClearXformOpOrder()
        bar_xf.AddTranslateOp().Set(Gf.Vec3d(float(sx), 0.0, float(bt + bh + handle_h)))

        apply_collider(bar.GetPrim())

# =====   바구니를 "움직이게" 만들기   =====
if BASKET_DYNAMIC:
    marker_prim = stage.GetPrimAtPath(TARGET_MARKER_PATH)
    UsdPhysics.RigidBodyAPI.Apply(marker_prim)

    mass_api = UsdPhysics.MassAPI.Apply(marker_prim)
    mass_api.CreateMassAttr(float(BASKET_MASS))

    rb = PhysxSchema.PhysxRigidBodyAPI.Apply(marker_prim)
    rb.CreateDisableGravityAttr(False)
    rb.CreateEnableCCDAttr(bool(BASKET_ENABLE_CCD))
    rb.CreateLinearDampingAttr(float(BASKET_LINEAR_DAMPING))
    rb.CreateAngularDampingAttr(float(BASKET_ANGULAR_DAMPING))
    rb.CreateSolverPositionIterationCountAttr(8)
    rb.CreateSolverVelocityIterationCountAttr(2)

    print("[Basket] Dynamic RigidBody enabled")

    return (x, y, z), "lime_green_basket", basket_rgb

# =====
# 메인 실행
# =====
def spawn_episode():
    """새 에피소드용 스폰"""

    # 물체 인덱스 로드
    objects_list = []
    if OBJECTS_INDEX_PATH.exists():
        with open(OBJECTS_INDEX_PATH, "r", encoding="utf-8") as f:
            for line in f:
                if line.strip():
                    try:
                        obj = json.loads(line)
                        if obj.get("status") == "ok" and obj.get("usd") and obj.get("name"):
                            objects_list.append(obj)
                    except Exception:
                        pass

    if not objects_list:
        print(f"[ERROR] 물체 인덱스 로드 실패: {OBJECTS_INDEX_PATH}")
        return None

```

```

# Physics 환경 보장 (Ground 는 만들지 않음)
ensure_physics_scene()
ensure_timeline_play()

print(f"\n{' '*50}")
print(f"   새 에피소드 스폰 ({len(objects_list)}개 물체 라이브러리)")
print(f"{' '*50}")

clear_spawned()
randomize_lighting()

obj_name, obj_pos, obj_scale = spawn_object(objects_list)
if obj_name is None:
    print("[ERROR] 물체 스폰 실패")
    return None

target_pos, target_name, target_rgb = spawn_target(avoid_pos=obj_pos)

print(f"\n   물체: {obj_name}")
print(f"   위치: ({obj_pos[0]:.3f}, {obj_pos[1]:.3f}, {obj_pos[2]:.3f})")
print(f"   스케일: {obj_scale:.2f} (고정)")

print(f"\n   타겟: 연두색 바구니 (Dynamic={BASKET_DYNAMIC})")
print(f"   위치: ({target_pos[0]:.3f}, {target_pos[1]:.3f}, {target_pos[2]:.3f})")

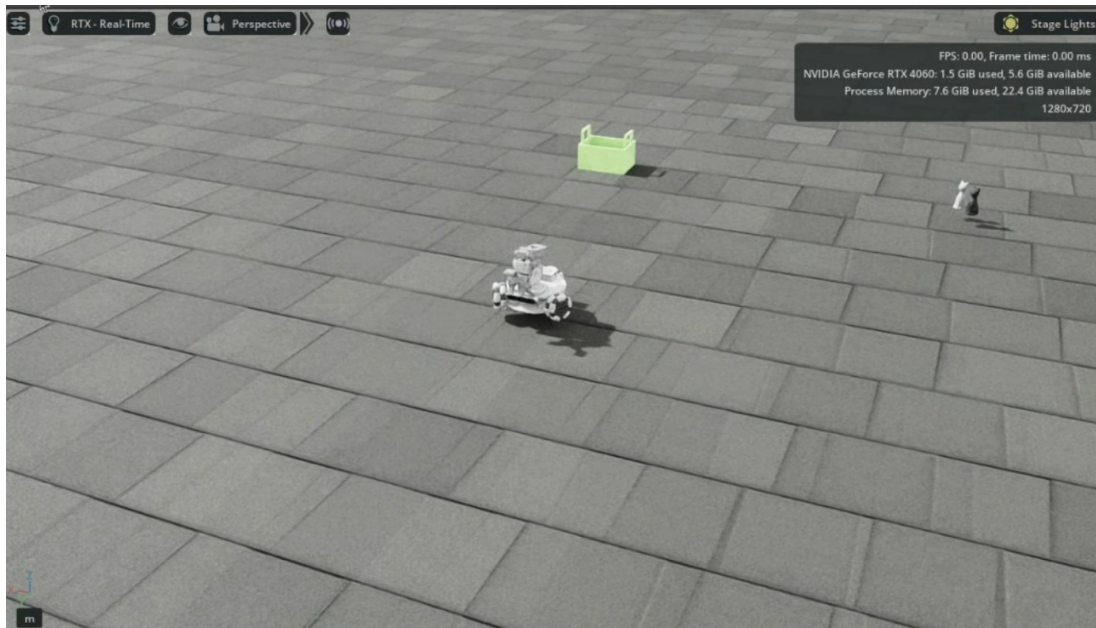
instruction = f"Pick up the {obj_name} and place it in the green basket"
print(f"\n   추천 instruction:")
print(f"   "{instruction}")

print(f"\n{' '*50}")
print(f"   스폰 완료!")
print(f"{' '*50 + "\n"}")

return {
    "object": {"name": obj_name, "position": obj_pos, "scale": obj_scale},
    "target": {"position": target_pos, "name": target_name, "color_rgb": target_rgb, "dynamic":
BASKET_DYNAMIC},
    "instruction": instruction,
}

# 실행
result = spawn_episode()

```



5.2 object filtering

generate_thumbnails.py (전체 코드)

```
#!/usr/bin/env python3
"""
Generate thumbnail images for 1030 MuJoCo scanned objects.
Uses matplotlib 3D rendering with texture colors.

Usage:
    pip install pillow numpy matplotlib
    python generate_thumbnails.py --catalog object_catalog_all.json --output thumbnails/
"""

import argparse
import json
import os
import sys
import time
import numpy as np

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from PIL import Image

BASE_DIR_CANDIDATES = [
    os.path.expanduser("~/isaac-objects/mujoco_scanned_objects/models"),
    "/home/yubin/isaac-objects/mujoco_scanned_objects/models",
    "/home/yubin11/isaac-objects/mujoco_scanned_objects/models",
    "/home/jovyan/isaac-objects/mujoco_scanned_objects/models",
]

def find_base_dir():
```



```

for d in BASE_DIR_CANDIDATES:
    if os.path.isdir(d):
        return d
return None

def load_obj_with_uv(path):
    """Load OBJ: vertices, UVs, and faces with UV indices."""
    verts = []
    uvs = []
    faces = [] # vertex indices (triangulated)
    face_uvs = [] # UV indices per face (triangulated)

    with open(path) as f:
        for line in f:
            line = line.strip()
            if line.startswith('v '):
                parts = line.split()
                verts.append([float(parts[1]), float(parts[2]), float(parts[3])])
            elif line.startswith('vt '):
                parts = line.split()
                uvs.append([float(parts[1]), float(parts[2])])
            elif line.startswith('f '):
                parts = line.split()[1:]
                v_idx = []
                uv_idx = []
                for p in parts:
                    chunks = p.split('/')
                    v_idx.append(int(chunks[0]) - 1)
                    if len(chunks) >= 2 and chunks[1]:
                        uv_idx.append(int(chunks[1]) - 1)
                # Triangulate
                for j in range(1, len(v_idx) - 1):
                    faces.append([v_idx[0], v_idx[j], v_idx[j+1]])
                    if len(uv_idx) == len(v_idx):
                        face_uvs.append([uv_idx[0], uv_idx[j], uv_idx[j+1]])

    return np.array(verts), np.array(uvs) if uvs else None, np.array(faces), np.array(face_uvs) if face_uvs
    else None

def sample_texture_colors(texture_img, uvs, face_uvs, faces):
    """Sample texture color at each face's center UV coordinate."""
    w, h = texture_img.size
    tex_array = np.array(texture_img) # H x W x 3/4

    n_faces = len(faces)
    colors = np.zeros((n_faces, 3), dtype=np.float64)

    if uvs is None or face_uvs is None or len(face_uvs) != n_faces:
        return None

    for i in range(n_faces):
        # Average UV of the 3 vertices
        uv_tri = uvs[face_uvs[i]] # (3, 2)
        uv_center = uv_tri.mean(axis=0)

        # UV to pixel (flip V)
        px = int(np.clip(uv_center[0] * (w - 1), 0, w - 1))
        py = int(np.clip((1.0 - uv_center[1]) * (h - 1), 0, h - 1))

        colors[i] = tex_array[py, px, :3] / 255.0

    return colors

```

```

def render_matplotlib(obj_dir, output_path, resolution=(640, 480)):
    """Render model.obj with texture.png colors using matplotlib."""
    mesh_path = os.path.join(obj_dir, "model.obj")
    tex_path = os.path.join(obj_dir, "texture.png")

    if not os.path.exists(mesh_path):
        return False

    try:
        verts, uvs, faces, face_uvs = load_obj_with_uv(mesh_path)
    except Exception:
        return False

    if len(verts) == 0 or len(faces) == 0:
        return False

    # Center and normalize
    center = (verts.max(axis=0) + verts.min(axis=0)) / 2
    verts -= center
    scale = np.max(verts.max(axis=0) - verts.min(axis=0))
    if scale > 0:
        verts /= scale

    # Subsample if too many faces
    max_faces = 8000
    if len(faces) > max_faces:
        idx = np.random.RandomState(42).choice(len(faces), max_faces, replace=False)
        faces = faces[idx]
        if face_uvs is not None and len(face_uvs) > 0:
            face_uvs = face_uvs[idx]

    polys = verts[faces]

    # Compute face normals for shading
    v0, v1, v2 = polys[:, 0], polys[:, 1], polys[:, 2]
    normals = np.cross(v1 - v0, v2 - v0)
    norms = np.linalg.norm(normals, axis=1, keepdims=True)
    norms[norms == 0] = 1
    normals /= norms

    # Light
    light_dir = np.array([0.4, -0.7, 0.6])
    light_dir /= np.linalg.norm(light_dir)
    intensity = np.abs(np.dot(normals, light_dir))
    intensity = 0.3 + 0.7 * intensity # ambient + diffuse

    # Get base colors from texture
    tex_colors = None
    if os.path.exists(tex_path):
        try:
            tex_img = Image.open(tex_path).convert('RGB')
            tex_colors = sample_texture_colors(tex_img, uvs, face_uvs, faces)
        except Exception:
            pass

    if tex_colors is None:
        # Fallback: warm gray
        base = np.tile([0.65, 0.62, 0.58], (len(faces), 1))
    else:
        base = tex_colors

    # Apply shading

```

```

face_colors = base * intensity[:, np.newaxis]
face_colors = np.clip(face_colors, 0, 1)
face_colors = np.column_stack([face_colors, np.ones(len(face_colors))])

# Plot
dpi = 100
fig = plt.figure(figsize=(resolution[0]/dpi, resolution[1]/dpi), dpi=dpi)
ax = fig.add_subplot(111, projection='3d')

pc = Poly3DCollection(polys, facecolors=face_colors, edgecolors='none', linewidths=0)
ax.add_collection3d(pc)

lim = 0.6
ax.set_xlim(-lim, lim)
ax.set_ylim(-lim, lim)
ax.set_zlim(-lim, lim)
ax.view_init(elev=25, azim=-60)

ax.set_facecolor('#141420')
fig.patch.set_facecolor('#141420')
ax.grid(False)
ax.set_axis_off()
plt.subplots_adjust(left=0, right=1, top=1, bottom=0)

fig.savefig(output_path, dpi=dpi, facecolor='#141420',
            bbox_inches='tight', pad_inches=0.05)
plt.close(fig)
return True

def render_placeholder(name, bbox, output_path, resolution=(640, 480)):
    img = Image.new('RGB', resolution, (20, 20, 32))
    from PIL import ImageDraw, ImageFont
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 22)
        font_sm = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 16)
    except Exception:
        font = ImageFont.load_default()
        font_sm = font
    cx, cy = resolution[0] // 2, resolution[1] // 2
    label = name.replace('_', ' ')[:45]
    draw.text((cx, cy - 30), label, fill=(180, 180, 180), font=font, anchor='mm')
    bx, by, bz = [round(v * 100, 1) for v in bbox]
    draw.text((cx, cy + 10), f"{bx} x {by} x {bz} cm", fill=(120, 120, 150), font=font_sm, anchor='mm')
    draw.text((cx, cy + 40), "rendering failed", fill=(100, 80, 80), font=font_sm, anchor='mm')
    img.save(output_path)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--catalog", default="object_catalog_all.json")
    parser.add_argument("--output", default="thumbnails")
    parser.add_argument("--resolution", type=int, nargs=2, default=[640, 480])
    parser.add_argument("--base_dir", default=None)
    parser.add_argument("--no_skip", action="store_true")
    parser.add_argument("--start", type=int, default=0)
    parser.add_argument("--end", type=int, default=-1)
    args = parser.parse_args()

    with open(args.catalog) as f:
        catalog = json.load(f)

    base_dir = args.base_dir or find_base_dir()

```

```

if base_dir is None:
    print("ERROR: Cannot find models directory.")
    print("Use --base_dir /path/to/mujoco_scanned_objects/models/")
    sys.exit(1)

test_name = catalog[0]["name"]
test_obj = os.path.join(base_dir, test_name, "model.obj")
test_tex = os.path.join(base_dir, test_name, "texture.png")
print(f"Base dir: {base_dir}")
print(f"model.obj exists: {os.path.exists(test_obj)}")
print(f"texture.png exists: {os.path.exists(test_tex)}")
if not os.path.exists(test_obj):
    print("ERROR: model.obj not found")
    sys.exit(1)

os.makedirs(args.output, exist_ok=True)
end = args.end if args.end > 0 else len(catalog)
items = catalog[args.start:end]
skip_existing = not args.no_skip

print(f"Rendering {len(items)} objects → {os.path.abspath(args.output)}/\n")

ok = fail = skip = 0
t0 = time.time()

for i, obj in enumerate(items):
    name = obj["name"]
    out_path = os.path.join(args.output, f"{name}.png")

    if skip_existing and os.path.exists(out_path):
        skip += 1
        continue

    obj_dir = os.path.join(base_dir, name)
    rendered = render_matplotlib(obj_dir, out_path, tuple(args.resolution))

    if rendered:
        ok += 1
    else:
        render_placeholder(name, obj["bbox"], out_path, tuple(args.resolution))
        fail += 1

total_done = ok + fail
if total_done % 50 == 0 and total_done > 0:
    elapsed = time.time() - t0
    rate = total_done / elapsed
    remaining = len(items) - i - 1
    eta = remaining / rate / 60 if rate > 0 else 0
    print(f" [{i+1}/{len(items)}] {elapsed:.0f}s ({rate:.1f}/s, ~{eta:.0f}min left) ✓{ok} ✗{fail} {skip}")

elapsed = time.time() - t0
print(f"\nDone in {elapsed:.0f}s — ✓{ok} ✗{fail} {skip}")
print(f"Open object_filter.html in browser")

if __name__ == "__main__":
    main()

```

첨부파일: object_catalog_all.json, object_filter.html



만약 이를 활용해 데이터 수집을 해야 한다면 1030 개는 너무 많기 때문에 이 중 활용할 물품을 선별하기 위한 ui 를 통해 object filtering 을 진행할 수 있다

```
# 폴더생성 및 파일 옮기기
mkdir -p ~/object_filter && cd ~/object_filter
cp object_filter.html generate_thumbnails.py object_catalog_all.json .
```

```
# 물품 사진 생성
pip install trimesh pyglet pillow numpy
python generate_thumbnails.py --catalog object_catalog_all.json --output thumbnails/
```

```
# 실행
xdg-open object_filter.html
```

